


**THE COPPERBELT UNIVERSITY**

**SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY**

**DEPARTMENT OF COMPUTER ENGINEERING**

# TOPIC: Multi-Source Energy-Harvesting Autonomous Sensor Node.

**GROUP SEVEN**


| STUDENT NUMBER | NAME |
|---|---|
| 22104657 | Misheck chipili mwiche |
| 23129470 | Lukundo mapalo |
| 22104946 | Fooko charles |
| 23128318 | Matongo mainga lushomo |


| SUPERVISOR | SIGNATURE | DATE |
|---|---|---|
| Mr. Shumba | ---------- | -----/-------/----- |

# Table of Contents

LIST OF FIGURES ..... 8
LIST OF ABBREVIATIONS ..... 9
ABSTRACT ..... 10
**CHAPTER ONE:** ..... 11
**Background of the Study** ..... 11
1.5 Scope of the Study ..... 16
1.6 Significance of the Study ..... 18
1.6.1 Academic Significance ..... 18
1.6.2 Industrial Significance ..... 19
1.6.3 Environmental Significance ..... 20
1.6.4 Economic Significance ..... 21
1.6.5 Societal Significance ..... 22
1.7 Research Questions ..... 23
Main Research Question ..... 23
Specific Research Questions ..... 23
1.8 Project Justification ..... 24
1.9 Organization of the Report ..... 25
1.10 Chapter Summary ..... 26
**CHAPTER TWO:** ..... 27
**LITERATURE REVIEW** ..... 27
1. Introduction ..... 27
2. Related Work (Academic Prior Studies) ..... 28
2.2 Ambient Energy Harvesting Techniques ..... 28
2.2.1 Solar Energy Harvesting ..... 28
2.2.2 Magnetic Field Energy Harvesting ..... 30
2.2.3 Radio Frequency (RF) Energy Harvesting ..... 31
2.2.4 Energy Storage Technologies for Energy Harvesting ..... 32
2.2.5 Kinetic and Aero-elastic Energy Harvesting: ..... 32
2.2.6 Cognitive Radio and Spectrum Efficiency: ..... 33
2.2.7 Check pointing for Intermittent Computing: ..... 33
2.2.8 Emerging Indoor Photovoltaics: ..... 33

2.2.9 Hybrid Energy Management and Machine Learning: 34
2.2.10 Computational RFID and Battery-Free Sensing (The WISP Project): 34
Previous Systems (or Similar Applications) 35
**3.1 EnOcean Device Management Platform** 35
Limitations: 36
Unresolved Problems: 36
Need for Current Research: 36
**3.2 Texas Instruments Energy Harvesting Evaluation GUI (BQ25570)** 37
Limitations: 38
Unresolved Problems: 38
Need for Current Research: 38
**3.3 Power cast RF Energy Harvesting Monitoring System** 39
Limitations: 40
Unresolved Problems: 40
Need for Current Research: 40
**3.4 E-peas Energy Harvesting Demonstration Software (AEM10941)** 41
Limitations: 42
Unresolved Problems: 42
Need for Current Research: 42
**3.5 Zolertia IoT Energy-Harvesting Node (Zolertia IPv6/6LoWPAN Ubidots client)** 43
Limitations: 44
Unresolved Problems: 44
Need for Current Research: 44
**3.6 Wireless Identification and Sensing Platform (WISP) Tools** 45
Limitations: 45
Unresolved Problems: 46
Need for Current Research: 46
**3.7 Nordic Semiconductor BLE Energy Profiling Tools** 47
Limitations: 47
Unresolved Problems: 48
Need for Current Research: 48
**3.8 Open Energy Monitor IoT Monitoring Platform** 49
Limitations: 49
Unresolved Problems**:** 50
Need for Current Research: 50

**3.9 Smart Mesh IP Power Monitoring Tools** ..... 51
Limitations: ..... 52
Unresolved Problems: ..... 52
Need for Current Research: ..... 52
**3.10 Things Board IoT Monitoring Platform** ..... 53
Limitations: ..... 53
Unresolved Problems: ..... 54
Need for Current Research: ..... 54
**4.** Lessons Learnt from the Review ..... 55
**5.** A Critique of the Review (Gaps, Inconsistencies, Omissions) ..... 56
2.16 Chapter Summary ..... 57
**CHAPTER THREE:** ..... 58
**SYSTEM DESIGN** ..... 58
3.1. Introduction ..... 58
3.2.System Objectives ..... 58
**3.3. RESEARCH METHODOLOGY** ..... 59
Iterative Engineering Process Model ..... 59
**3.2.1 Stage One: Requirement Analysis** ..... 60
System Requirements ..... 60
**3.2.2 Stage Two: System Architecture Design and Modeling** ..... 61
3.2.2 Overview System Architecture ..... 62
**3.2.3 Stage Three: Hardware Design and Component Selection** ..... 63
**3.2.3.1 Electrical Design Specifications** ..... 63
3.2.3.2 Energy Harvesting Subsystem ..... 64
3.2.3.2.1 Solar Harvesting Section ..... 64
FIGURE 3.2.3.2.1: Solar Harvesting Section ..... 64
**Photovoltaic Panel** ..... 64
**DC-DC Converter** ..... 64
3.2.3.2.2 Magnetic Harvesting Section ..... 65
FIGURE 3.2.3.2.2: Magnetic Harvesting Section (a) ..... 65
FIGURE 3.2.3.2.2: Magnetic Harvesting Section (b) ..... 65
**Magnetic Harvester (Coil + Core)** ..... 65
**Rectifier & Filtering Network** ..... 65
3.2.3.2.3 RF Harvesting Section ..... 66

FIGURE: 3.2.3.2.3 RF Harvesting Section ..... 66
**RF Antenna & RF Rectifier Circuit** ..... 66
**Filter Capacitors** ..... 66
3.2.3.2.4 Power Management and Storage Subsystem ..... 67
**Figure 3.2.3.2.4: Energy Storage Circuit** ..... 67
3.2.3.2.5 Central Power Management Unit ..... 68
3.2.3.2.5.1 Super capacitor Energy Storage ..... 68
3.2.3.2.5.2 Voltage Divider + ADC Monitoring ..... 68
**3.2.3.2.6 Energy Storage Calculation** ..... 69
**Super capacitor Energy Analysis** ..... 69
3.2.3.2.7 Processing and Sensing Subsystem ..... 70
3.2.3.2.7.1 Microcontroller & Sensors ..... 71
Power Budget Analysis ..... 71
3.2.3.2.8 Serial Communication and Data Logging Subsystem ..... 72
FIGURE: 3.2.3.2.8 Serial Communication and Data Logging Subsystem ..... 72
**Figure 3.2.3.2.8: Data flow Diagram** ..... 72
3.2.3.2.8 USB-UART INTERFACE ..... 73
Overall Energy & Signal Flow ..... 73
**Power Path:** ..... 73
**Data Path:** ..... 73
**3.2.3.2.8.1 System Behavior Summary** ..... 74
**3.2.4 Stage Four: Firmware Development and Energy-Aware Control Algorithms** ..... 75
**Figure 3.2.4: Firmware operational flowchart for Energy-Aware Duty Cycling** ..... 75
3.2.4.1 Software Architecture ..... 76
**3.2.4.1.1 Energy Monitoring Module:** ..... 76
**3.2.4.1.2 Adaptive Duty Cycling Module:** ..... 76
**3.2.4.1.3 Sensor Management Module:** ..... 76
**3.2.4.1.4Serial Transmission Module** ..... 76
3.2.4.2 Telemetry Data Format ..... 77
**3.2.4.3 Duty Cycle Timing Table** ..... 77
3.2.4.4Laptop Logging Application ..... 78
3.2.4.5 System Operation Flow ..... 78
Advantages of the Selected Architecture ..... 79
Disadvantages of the Selected Architecture ..... 80

**Failure Modes and Mitigation** ..... 80
**Risk Analysis** ..... 80
3.3 Tools and Technologies to Be Used ..... 81
3.11.15 Chapter Summary ..... 82
**CHAPTER FOUR:** ..... 83
**SYSTEM IMPLEMENTATION, TESTING AND PERFORMANCE EVALUATION** ..... 83
4.1 Introduction ..... 83
4.2 Prototype Implementation ..... 84
4.2.1 Hardware Prototype ..... 84
4.2.2 Hardware Integration ..... 85
4.3 Firmware Implementation ..... 85
4.4 Laptop Logging Application ..... 86
4.5 System Testing Strategy ..... 86
4.5.1 Unit Testing ..... 87
**Solar Harvesting Test** ..... 87
**Magnetic Harvester Test** ..... 87
**RF Harvester Test** ..... 88
**Super capacitor Test** ..... 88
4.5.2 Firmware Testing ..... 89
Test Cases ..... 89
4.5.3 Communication Testing ..... 89
Procedure ..... 89
4.5.4 Sensor Validation ..... 90
Parameters Evaluated ..... 90
4.6 Performance Evaluation ..... 90
4.6.1 Evaluation Metrics ..... 91
4.7 Sample Experimental Results ..... 91
4.8 Discussion of Results ..... 92
4.9. Achievement of the Research Objectives ..... 93
**Objective 1:** To analyze suitable ambient energy harvesting techniques. ..... 93
**Objective 2:** To design an efficient power management and storage subsystem. ..... 93
**Objective 3:** To implement adaptive duty-cycling algorithms. ..... 93
**Objective 4:** To develop a telemetry logging system for evaluating system performance ..... 94

**4.10 Chapter Summary** ..... 95
**CHAPTER FIVE:** ..... 96
**CONCLUSIONS AND RECOMMENDATIONS** ..... 96
5.1 **Major Contributions of the Study** ..... 96
5.2 **Limitations of the Study** ..... 97
5.3 **Recommendations for Future Work** ..... 98
Wireless Communication ..... 98
Machine Learning for Energy Prediction ..... 98
Additional Harvesting Techniques ..... 98
Additional Environmental Sensors ..... 99
Printed Circuit Board Optimization ..... 99
Internet of Things Integration ..... 99
**5.4 Summary of the Study** ..... 100
**5.5 References** ..... 102
**5.6 Appendix** ..... 104
**Firmware Code:** ..... 104
**Laptop Py. Logger_File:** ..... 114

# LIST OF FIGURES


- Figure 2.1 Wireless Sensor Node Architecture
- Figure 2.2 Solar Energy Harvesting Principle
- Figure 2.3 Magnetic Energy Harvesting Principle
- Figure 2.4 RF Energy Harvesting Principle
- Figure 2.5 Conceptual Framework
- Figure 3.1 Overall System Architecture
- Figure 3.2 Hardware Architecture
- Figure 3.3 Energy Harvesting Block Diagram
- Figure 3.4 Power Management Circuit
- Figure 3.5 Voltage Divider Circuit
- Figure 3.6 Firmware Flowchart
- Figure 3.7 Activity Diagram
- Figure 3.8 Sequence Diagram
- Figure 3.9 State Machine Diagram
- Figure 3.10 Software Architecture
- Figure 4.1 Hardware Prototype
- Figure 4.2 Serial Communication Flow
- Figure 4.3 Experimental Setup
- Figure 4.4 Telemetry Data Flow

- Table 2.1 Comparative Analysis of Existing Systems
- Table 3.1 Electrical Design Specifications
- Table 3.2 Adaptive Duty-Cycle Timing
- Table 3.3 UART Communication Parameters
- Table 3.4 Telemetry Packet Structure
- Table 3.5 Power Budget Analysis
- Table 4.1 Hardware Components
- Table 4.2 Unit Testing Results
- Table 4.3 Integration Testing Results

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| ADC | Analog-to-Digital Converter |
| AC | Alternating Current |
| BLE | Bluetooth Low Energy |
| CSV | Comma-Separated Values |
| DC | Direct Current |
| DFD | Data Flow Diagram |
| IDE | Integrated Development Environment |
| IoT | Internet of Things |
| MCU | Microcontroller Unit |
| MPPT | Maximum Power Point Tracking |
| PCB | Printed Circuit Board |
| PMU | Power Management Unit |
| PV | Photovoltaic |
| RF | Radio Frequency |
| UART | Universal Asynchronous Receiver-Transmitter |
| USB | Universal Serial Bus |
| WSN | Wireless Sensor Network |

# ABSTRACT

The rapid growth of Wireless Sensor Networks (WSNs) and Internet of Things (IoT) technologies has increased the demand for autonomous sensing systems capable of operating for extended periods without frequent battery replacement. Conventional battery-powered sensor nodes suffer from limited operational lifetime, periodic maintenance, and environmental concerns associated with battery disposal. These challenges are particularly significant in remote and hazardous environments where battery replacement is difficult, expensive, or impractical.

This study presents the design and implementation of a **Multi-Source Energy-Harvesting Autonomous Sensor Node** that combines solar, magnetic, and radio-frequency (RF) energy harvesting techniques to provide a sustainable energy supply for low-power embedded sensing applications. The proposed system integrates an energy harvesting subsystem, power management and super capacitor-based energy storage subsystem, embedded processing and sensing subsystem, and a serial communication subsystem supported by a Python-based telemetry logging application.

Adaptive energy management is achieved through firmware that continuously monitors the stored energy level and dynamically adjusts sensing frequency using an adaptive duty-cycling algorithm. Environmental measurements are transmitted through a USB-UART interface to a laptop, where telemetry data is validated, displayed, and recorded in CSV format for performance evaluation.

The proposed architecture demonstrates the feasibility of integrating multiple ambient energy sources within a single embedded platform while reducing dependence on conventional batteries. The modular system architecture provides a flexible foundation for future research involving wireless communication, additional harvesting techniques, intelligent energy prediction, and large-scale Internet of Things deployments.

**Keywords:** Energy Harvesting, Embedded Systems, Wireless Sensor Networks, Internet of Things, Super capacitor, Adaptive Duty Cycling, Renewable Energy, Power Management, Autonomous Sensor Node.

# CHAPTER ONE:

## Background of the Study

The rapid advancement of embedded systems, wireless sensor networks (WSNs), and the Internet of Things (IoT) has transformed the way data is collected, processed, and utilized across various sectors of society. Modern sensor nodes are increasingly being deployed in applications such as environmental monitoring, precision agriculture, industrial automation, structural health monitoring, mining, military surveillance, and smart city infrastructure. These applications depend heavily on continuous sensing and reliable communication in order to provide accurate and real-time information for decision-making. Despite these technological advancements, one major limitation continues to affect the widespread deployment of autonomous sensor systems: energy availability.

Traditional wireless sensor nodes rely primarily on electrochemical batteries as their sole source of power. While batteries provide a stable supply of electrical energy, they possess several inherent limitations. Batteries have a finite lifespan, require periodic replacement or recharging, degrade over time due to repeated charging cycles, and present environmental concerns during disposal. In large-scale deployments involving hundreds or thousands of sensor nodes, maintenance costs become significantly high because every depleted battery requires physical replacement. These challenges become even more severe in inaccessible environments such as underground mining tunnels, remote forests, high-voltage electrical substations, bridges, pipelines, and hazardous industrial locations where regular maintenance is either difficult or impossible. As the number of deployed sensor nodes continues to increase, battery maintenance becomes economically unsustainable and technically impractical.

To overcome these limitations, researchers have increasingly focused on **energy harvesting**, a technique through which electronic devices extract usable electrical energy from naturally available environmental sources. Instead of relying solely on stored battery energy, an energy-harvesting system continuously captures ambient energy and converts it into electrical power capable of operating embedded electronics. Common ambient energy sources include solar radiation,

electromagnetic fields, radio frequency (RF) signals, thermal gradients, vibration, wind, and mechanical movement. By harvesting energy directly from the surrounding environment, sensor nodes can significantly extend their operational lifetime while reducing dependence on battery replacement.

Among the available harvesting techniques, solar energy harvesting has received considerable attention due to its relatively high power density and technological maturity. Photovoltaic cells convert incident sunlight into electrical energy with comparatively high efficiency under favorable lighting conditions. However, solar energy remains highly dependent on environmental conditions. During nighttime, cloudy weather, indoor environments, or locations with heavy shading, the harvested power decreases significantly, making continuous operation difficult. Consequently, systems that depend solely on photovoltaic harvesting cannot guarantee uninterrupted sensing throughout varying environmental conditions.

Magnetic field energy harvesting provides another alternative for supplying energy to autonomous sensor systems. Alternating magnetic fields surrounding transformers, electric motors, transmission lines, and industrial equipment induce electrical current within specially designed coils according to Faraday's Law of Electromagnetic Induction. Unlike solar energy, magnetic energy harvesting is largely independent of lighting conditions and can provide continuous energy in industrial environments where electrical machinery operates continuously. Nevertheless, harvested power remains relatively small and depends greatly on magnetic field strength, coil geometry, and operating frequency.

Radio Frequency (RF) energy harvesting has also emerged as an attractive technique for ultra-low-power electronic systems. RF harvesting captures electromagnetic waves transmitted from Wi-Fi routers, television stations, mobile communication towers, RFID readers, and other wireless communication equipment. Using antennas and high-efficiency rectifier circuits, these radio signals are converted into direct current capable of charging energy storage devices. Although ambient RF energy is almost always present in populated environments, its power density is extremely low, making it unsuitable as the sole energy source for sensor operation. Recent developments therefore emphasize **multi-source energy harvesting**, where several independent harvesting mechanisms operate simultaneously. Rather than depending on only one environmental source, multiple

harvesters complement each other, thereby increasing overall system reliability. During daytime, solar energy may become the dominant source; inside industrial facilities, magnetic harvesting can continue supplying energy; while RF harvesting contributes supplementary energy whenever wireless signals are available. Such hybrid systems substantially improve energy availability under changing environmental conditions and reduce the probability of complete power loss.

Efficient utilization of harvested energy further requires intelligent energy management. Since ambient energy availability fluctuates continuously, the proposed system stores harvested energy within a super capacitor instead of a conventional rechargeable battery. Super capacitors provide rapid charging capability, extremely long operational life, high charge-discharge efficiency, and the ability to withstand millions of charging cycles without significant degradation. The stored energy is continuously monitored by the microcontroller through an Analog-to-Digital Converter (ADC). Based on the measured energy level, the embedded firmware dynamically adjusts sensing frequency, communication intervals, and operational modes using adaptive duty cycling. Under low-energy conditions, the node enters sleep mode to conserve energy; when energy availability improves, normal sensing resumes. This adaptive behavior enables energy-neutral operation while maximizing system lifetime.

Unlike many existing research prototypes that emphasize cloud connectivity and wireless communication, the proposed system deliberately adopts a simplified experimental architecture in which telemetry data is transmitted through a USB-UART interface to a laptop. A Python-based logging application receives serial data, timestamps each measurement, and stores sensor information in comma-separated value (CSV) format for subsequent analysis. This architecture reduces communication complexity, minimizes firmware overhead, eliminates wireless transmission losses, and allows accurate evaluation of the adaptive energy management algorithms under controlled laboratory conditions. Consequently, this project proposes the design and implementation of a **Multi-Source Energy-Harvesting Autonomous Sensor Node** capable of harvesting energy from solar, magnetic, and radio-frequency sources while intelligently managing available energy to support autonomous environmental sensing. By combining multiple harvesting technologies with adaptive energy management and efficient data logging, the system aims to demonstrate a practical approach toward maintenance-free sensor deployment suitable for future Internet of Things applications.

## 1.2 Problem Statement:

Wireless sensor networks continue to play an increasingly important role in monitoring physical environments across agriculture, mining, industrial automation, environmental protection, infrastructure monitoring, and military applications. Despite their widespread use, most commercially available sensor nodes continue to depend on conventional batteries as their primary energy source. Battery-powered systems suffer from limited operational lifetime, periodic maintenance requirements, increasing replacement costs, and environmental pollution associated with battery disposal. These limitations become increasingly significant when sensor nodes are deployed in remote or hazardous environments where routine maintenance is either expensive or physically impossible.

Several energy-harvesting techniques have been developed to address these challenges; however, most existing implementations depend on only a single ambient energy source such as solar radiation, radio frequency signals, or magnetic fields. Since environmental energy sources are inherently variable, single-source harvesting systems frequently experience power interruptions that reduce sensing reliability and shorten operational periods. In addition, many existing systems employ fixed sensing schedules that fail to adapt to fluctuating energy availability, resulting in inefficient energy utilization and unnecessary power consumption.

Therefore, there is a need to develop an autonomous sensor node capable of harvesting energy from multiple ambient sources while intelligently managing available energy through adaptive duty cycling and efficient power management. Such a system would improve reliability, reduce maintenance requirements, and support sustainable long-term sensor deployment.

## 1.3 Aim of the Study

The aim of this study is to design and implement a multi-source energy-harvesting autonomous sensor node capable of harvesting ambient energy from solar, magnetic, and radio-frequency sources while intelligently managing available energy for reliable autonomous sensing and telemetry data acquisition.

## 1.4 Specific Objectives

The objectives of this project are to:

1. Analyze suitable ambient energy harvesting techniques for autonomous sensor systems.
2. Design an efficient multi-source energy harvesting subsystem integrating solar, magnetic, and RF harvesters.
3. Develop an intelligent power management subsystem capable of regulating harvested energy and storing it in a super capacitor.
4. Implement adaptive duty-cycling algorithms that dynamically adjust sensing and communication according to available energy.
5. Design a serial telemetry logging system using USB-UART communication and a Python-based laptop logger for performance evaluation.

## 1.5 Scope of the Study

The scope of this project is limited to the design, implementation, testing, and evaluation of a prototype **Multi-Source Energy-Harvesting Autonomous Sensor Node** capable of operating using multiple ambient energy sources. The project concentrates on demonstrating the feasibility of combining solar energy, magnetic field energy, and radio frequency (RF) energy harvesting within a single embedded system to improve energy availability and operational reliability. The system is intended for low-power sensing applications where continuous battery replacement is undesirable or impractical.

The proposed prototype consists of five major subsystems: the energy harvesting subsystem, the power management subsystem, the energy storage subsystem, the processing and sensing subsystem, and the serial communication and data logging subsystem. These subsystems work together to harvest energy from the environment, regulate and store the harvested energy, monitor energy availability, collect environmental data, and transmit telemetry information to a laptop for storage and analysis.

The harvesting subsystem integrates three independent energy sources:

- A photovoltaic (PV) panel for harvesting solar energy.
- An inductive magnetic harvesting circuit for extracting energy from alternating magnetic fields.
- An RF harvesting circuit for capturing ambient radio-frequency energy.

Each energy source is connected to dedicated conditioning circuitry before converging into a common power management unit responsible for voltage regulation and safe energy storage within a super capacitor. This architecture increases system resilience by ensuring that the temporary absence of one energy source does not completely interrupt system operation.

Unlike many Internet of Things (IoT) research projects that employ wireless cloud communication, this project intentionally adopts a USB-UART serial communication interface. Environmental data collected by the sensor node is transmitted directly to a laptop running a Python-based logging

application. This approach simplifies implementation, minimizes communication overhead, improves experimental repeatability, and allows accurate observation of energy-aware system behavior.

The sensing subsystem is limited to monitoring environmental parameters supported by the selected sensors, particularly temperature and humidity. Although the proposed architecture can be extended to support additional sensing devices such as pressure, gas concentration, vibration, light intensity, or soil moisture sensors, these extensions fall outside the scope of the current research.

Similarly, the project does not include:

- Internet connectivity
- Cloud databases
- Mobile applications
- Wireless communication protocols such as Wi-Fi, ZigBee, Bluetooth Low Energy (BLE), LoRaWAN, or NB IoT
- Machine learning prediction algorithms
- Commercial PCB mass production

These advanced features represent opportunities for future work after validating the core concept of adaptive multi-source energy harvesting. The study primarily focuses on proving that intelligent energy management, adaptive duty cycling, and hybrid energy harvesting can significantly improve the operational sustainability of autonomous embedded sensor nodes.

## 1.6 Significance of the Study

The importance of this research extends beyond the implementation of a single prototype. It contributes to ongoing efforts aimed at reducing dependence on conventional battery-powered embedded systems while promoting sustainable and energy-efficient technologies.

### 1.6.1 Academic Significance

From an academic perspective, the project contributes to the growing body of research in embedded systems engineering, wireless sensor networks, energy harvesting, and low-power electronics. It demonstrates how several engineering disciplines—including electronics, embedded programming, power electronics, computer engineering, and software development—can be integrated into a single intelligent autonomous system.

The project also provides practical experience in applying engineering principles such as system modelling, hardware-software co-design, energy budgeting, power regulation, firmware development, and system evaluation. The resulting prototype can serve as a useful teaching and research platform for future undergraduate and postgraduate studies in computer engineering and related disciplines.

## 1.6.2 Industrial Significance

Industries increasingly rely on continuous monitoring of equipment, production processes, and environmental conditions. Examples include:

- Mining operations
- Manufacturing plants
- Electrical substations
- Oil and gas facilities
- Smart factories

Deploying battery-powered sensors in such environments often results in significant maintenance costs due to frequent battery replacement.

The proposed system reduces these maintenance requirements by continuously harvesting energy from surrounding environmental sources. Since industrial environments naturally contain magnetic fields, RF signals, and sometimes sufficient lighting, the proposed hybrid harvesting architecture becomes particularly suitable for industrial monitoring applications.

### 1.6.3 Environmental Significance

One of the major environmental challenges associated with conventional embedded systems is electronic waste generated by discarded batteries.

Rechargeable batteries contain chemical compounds that require careful disposal to avoid environmental contamination. Large-scale sensor deployments may require thousands of battery replacements throughout their operational lifetime.

The proposed battery-less architecture minimizes battery dependence by replacing conventional chemical storage with super capacitor technology, which offers:

- Millions of charging cycles
- Rapid charging
- Minimal maintenance
- Longer operational lifespan
- Reduced environmental impact

Consequently, the proposed system contributes toward greener and more sustainable embedded technologies.

## 1.6.4 Economic Significance

Battery replacement represents a major portion of operational costs in wireless sensor deployments.

For example, an organization operating 5,000 remote sensor nodes may spend considerable financial resources on:

- purchasing replacement batteries,
- transportation,
- maintenance personnel,
- downtime during battery replacement.

By harvesting freely available ambient energy, the proposed system significantly reduces long-term maintenance costs.

Although the initial implementation cost may be slightly higher because of multiple harvesting modules, the reduction in maintenance expenses makes the overall solution economically attractive over extended operational periods.

### 1.6.5 Societal Significance

Autonomous sensor technologies increasingly support public services including:

- environmental monitoring,
- disaster management,
- smart agriculture,
- weather observation,
- infrastructure safety,
- public health monitoring.

Reliable energy-autonomous sensors enable continuous monitoring even in remote regions where conventional electrical infrastructure is unavailable.

Such technologies can assist governments, researchers, engineers, farmers, and environmental agencies in making informed decisions based on continuously available environmental information.

# 1.7 Research Questions

The study seeks to answer the following research questions:

## Main Research Question

How can multiple ambient energy harvesting techniques be integrated to develop an autonomous sensor node capable of intelligent energy management and reliable long-term operation?

## Specific Research Questions

1. Which ambient energy harvesting techniques are most suitable for autonomous sensor systems?
2. How can solar, magnetic, and RF energy harvesting be effectively integrated into one embedded platform?
3. What power management techniques are required to regulate energy harvested from multiple independent sources?
4. How can adaptive duty cycling improve energy utilization under varying environmental conditions?
5. How effective is USB-UART telemetry logging for evaluating energy-aware embedded systems?

## 1.8 Project Justification

The motivation for this research arises from limitations observed in existing battery-powered wireless sensor nodes.

Existing systems reviewed during the literature survey demonstrate several shortcomings, including:

- dependence on a single energy harvesting source,
- fixed sensing intervals,
- limited adaptive behavior,
- battery dependence,
- high maintenance costs,
- reduced operational lifetime.

The literature also indicates that while commercial energy harvesting platforms provide efficient harvesting hardware, most lack intelligent firmware capable of adapting sensing behavior according to real-time energy availability. Similarly, many platforms focus on evaluating individual harvesting technologies rather than integrating several complementary energy sources within one adaptive architecture.

The proposed system addresses these limitations through:

- hybrid energy harvesting,
- intelligent power management,
- adaptive sensing,
- super capacitor energy storage,
- energy-aware firmware,
- simplified serial telemetry monitoring.

This combination aims to produce a more reliable and sustainable autonomous sensing platform.

## 1.9 Organization of the Report

This report is organized into five chapters.

**Chapter One:** introduces the research by presenting the background, problem statement, objectives, scope, significance, research questions, and project justification.

**Chapter Two:** presents a comprehensive literature review of existing energy harvesting technologies, previous research studies, commercial energy harvesting platforms, and identifies research gaps that justify the proposed work.

**Chapter Three:** discusses the complete system analysis and design, including system requirements, architecture, subsystem modelling, circuit design, hardware selection, software architecture, mathematical modelling, engineering calculations, power budget analysis, and adaptive energy management algorithms.

**Chapter Four:** presents the implementation of both hardware and software components, prototype development, firmware implementation, Python logging application, testing procedures, performance evaluation, experimental results, and discussion.

**Chapter Five:** concludes the research by summarizing the major findings, discussing project limitations, providing recommendations, and suggesting future areas for further investigation.

## 1.10 Chapter Summary

This chapter introduced the concept of autonomous energy-harvesting sensor systems and established the motivation for developing a multi-source energy-harvesting platform. The discussion highlighted the limitations associated with battery-powered wireless sensor nodes and explained how integrating multiple ambient energy sources, intelligent power management, and adaptive duty-cycling can improve long-term operational reliability. The chapter also defined the research objectives, scope, significance, research questions, and justification for the study.

Having established the research context, the next chapter reviews previous academic studies, existing commercial systems, ambient energy harvesting technologies, and identifies the research gaps that the proposed **Multi-Source Energy-Harvesting Autonomous Sensor Node** seeks to address.

# CHAPTER TWO:

# LITERATURE REVIEW

## 1. Introduction

The rapid growth of Wireless Sensor Networks (WSNs) and Internet of Things (IoT) applications in areas such as environmental monitoring, smart agriculture, industrial automation, and infrastructure surveillance has intensified the demand for long-term, maintenance-free sensor operation. Traditionally, most sensor nodes rely on batteries as their primary power source. While batteries provide predictable energy supply, they introduce major limitations, including limited operational lifetime, frequent maintenance, high replacement costs, and environmental concerns related to battery disposal. These challenges are particularly significant in remote, hazardous, or large-scale deployments where physical access to sensor nodes is difficult or costly.

Energy harvesting has emerged as a promising alternative to battery-powered sensor nodes by enabling devices to extract energy from ambient sources such as solar radiation, magnetic fields, and radio frequency (RF) signals. However, many early energy harvesting systems relied on a single ambient source, making them vulnerable to environmental variability and intermittency. As a result, such systems often fail to guarantee reliable and continuous operation. This has motivated research into multi-source energy harvesting combined with efficient power management and adaptive duty-cycling mechanisms. This chapter reviews prior academic studies and existing systems to identify their strengths, limitations, and unresolved challenges, thereby justifying the need for the proposed multi-source energy-harvesting autonomous sensor node.

## 2. Related Work (Academic Prior Studies)

Early research in ambient energy harvesting demonstrated the feasibility of powering low-power electronic devices using scavenged environmental energy. Foundational studies established that micro-watt to milli-watt power levels could support basic sensing, processing, and communication tasks if energy consumption was carefully managed. This led to the concept of energy-neutral operation, where a sensor node dynamically adapts its workload to match harvested energy availability.

Because all ambient energy sources are inherently intermittent, energy storage plays a critical role in autonomous sensor systems. While rechargeable batteries provide high energy density, their limited cycle life and aging effects reduce suitability for long-term maintenance-free deployment. Super capacitors have therefore gained increasing attention due to their fast charging capability, long cycle life, and compatibility with battery-less energy harvesting architectures.

## 2.2 Ambient Energy Harvesting Techniques

Research by Paradiso and Sterner demonstrated the feasibility of powering low-power electronics using ambient energy. Solar, piezoelectric, and thermoelectric harvesting techniques have been widely explored, though each suffers from environmental dependency namely.

### 2.2.1 Solar Energy Harvesting

Solar energy harvesting has been the most extensively studied technique due to its relatively high power density and technological maturity. Photovoltaic cells can generate significant power under favorable lighting conditions; however, their output is highly dependent on irradiance, temperature, and panel orientation. To address these variations, Maximum Power Point Tracking (MPPT) techniques are widely employed to maximize energy extraction. Despite these improvements, solar harvesting remains intermittent and unreliable during nighttime or poor weather conditions.

Solar energy harvesting is one of the most extensively researched and practically deployed ambient energy harvesting techniques due to its comparatively high energy density and technological maturity. It employs photovoltaic (PV) cells to convert incident solar radiation directly into electrical energy through the photovoltaic effect **(Paradiso & Starner, 2025)**. Because solar radiation provides higher power levels than most other ambient sources, PV harvesting is frequently adopted as the primary energy source for outdoor and semi-outdoor wireless sensor systems **(Roundy, Wright, & Rabaey, 2024).**

Experimental and analytical studies indicate that PV-based harvesters can produce output power ranging from a few microwatts under indoor or low-light conditions to several milli-watts under direct sunlight, depending on illumination intensity, panel surface area, material efficiency, and temperature **(Roundy et al., 2004)**. However, the electrical characteristics of PV cells are highly nonlinear. Variations in irradiance and temperature significantly affect output voltage and current, making direct coupling to electronic loads inefficient and potentially unstable.

To mitigate these effects, Maximum Power Point Tracking (MPPT) techniques are widely employed to dynamically adjust the operating point of the PV cell and maximize harvested energy under changing environmental conditions **(Esram & Chapman, 2007).** Despite these advancements, solar energy harvesting remains inherently intermittent, as it depends on daylight cycles and weather conditions, limiting its reliability as a standalone energy source **(Paradiso & Starner, 2025).**

### 2.2.2 Magnetic Field Energy Harvesting

Magnetic field energy harvesting exploits alternating magnetic fields generated by electrical infrastructure such as power transmission lines, transformers, motors, and industrial machinery **(Beeby & White, 2020)**. This technique typically utilizes inductive coils to convert time-varying magnetic flux into electrical energy based on Faraday's law of electromagnetic induction (Roundy, **Wright, & Rabaey, p. 2020)**.

Magnetic energy harvesting is particularly advantageous in indoor and industrial environments where strong magnetic fields are continuously present. Unlike solar harvesting, it is independent of lighting conditions and can provide energy as long as the source equipment remains operational (Beeby & White, 2020). This makes it attractive for environments where light availability is limited or inconsistent.

However, the harvested power levels are generally low and strongly dependent on coil geometry, distance from the magnetic source, operating frequency, and field strength **(Roundy, Wright, & Rabaey, 2024)**. As a result, magnetic harvesting alone is insufficient to sustain continuous sensing and wireless communication. Research indicates that it is most effective as a supplementary energy source when combined with efficient rectification, voltage boosting, and intelligent energy management circuits (Beeby & White, 2020).

### 2.2.3 Radio Frequency (RF) Energy Harvesting

Radio Frequency (RF) energy harvesting captures energy from ambient electromagnetic waves emitted by radio transmitters, television stations, cellular base stations, and Wi-Fi access points (Paradiso & Starner, 2025, p. 2022)**.** A typical RF harvesting system consists of an antenna, impedance matching network, rectifier, and energy storage element, commonly referred to as antenna. **(Sample, Yeager, Powledge, Mamishew, & Smith, p. 2021).**

One advantage of RF harvesting is its availability in both indoor and outdoor environments, particularly in urban areas where RF signals are continuously present. However, the ambient RF power density is extremely low, often in the Nano-watt to microwatt range, making RF harvesting unsuitable as a primary energy source for most sensor applications (Roundy, Wright, & Rabaey, 2024, p. 2022).

Despite these limitations, RF harvesting plays an important role in battery-less and ultra-low-power systems. It is particularly effective for standby operation, periodic wake-up events, and trickle charging of energy storage components when combined with high-efficiency rectification and impedance matching **(Sample, Yeager, Powledge, Mamishew, & Smith, p. 2021).** Consequently, RF harvesting is most effective when integrated into multi-source energy harvesting architectures.

## 2.2.4 Energy Storage Technologies for Energy Harvesting

Energy storage is a fundamental component of energy harvesting systems due to the intermittent and unpredictable nature of ambient energy sources **(Priya & Inman, 2009)**. The primary function of storage is to buffer harvested energy and provide a stable and continuous power supply to electronic loads during periods of insufficient energy availability.

Rechargeable batteries, such as lithium-ion and lithium-polymer types, offer high energy density and support long-duration operation. However, they suffer from limited charge discharge cycles, aging effects, leakage, and environmental concerns, which reduce their suitability for long-term, maintenance-free deployments **(Beeby & White, p. 2024)**.

Super capacitors, also known as ultra-capacitors, provide high power density, rapid charging capability, and extremely long cycle life, often exceeding millions of cycles without significant degradation **(Priya & Inman, p. 2020)**. These characteristics make them particularly suitable for battery-less energy harvesting systems. Consequently, super capacitors and thin-film storage devices are increasingly preferred, with power management integrated circuits (PMICs) playing a critical role in voltage regulation, energy protection, and system stability (Beeby & White, p. 2020).

## 2.2.5 Kinetic and Aero-elastic Energy Harvesting:

Recent research has moved beyond simple vibration to exploit aero-elastic flutter for industrial and aerospace applications. By using piezoelectric materials (such as PZT-5A) attached to wing-like structures, researchers have demonstrated the ability to generate up to **10.8mW** of power from airflow far exceeding traditional vibration harvesters **(Erturk, Vieira, De Marqui, & Inman, 2010)**. This provides a critical solution for monitoring remote infrastructure like wind turbines or aircraft sensors where solar and thermal gradients are inconsistent.

## 2.2.6 Cognitive Radio and Spectrum Efficiency:

A major bottleneck in Energy Harvesting WSNs (EH-WSNs) is the energy wasted during "spectrum sensing" and data collisions. Modern studies integrate Cognitive Radio (CR) to allow energy-neutral nodes to automatically detect and utilize unused spectrum bands. This "Energy-Aware Spectrum Management" ensures that nodes only transmit when a clear channel is found, preventing the wasted energy of re-transmissions a vital strategy for maintaining the stability of the energy buffer in low-power environments **(Akan, Karli, & Ergul, p. 2023)**.

## 2.2.7 Check pointing for Intermittent Computing:

Since ambient energy is often highly transient, a significant body of recent work focuses on **state-check pointing mechanisms**. Unlike traditional sensors that restart their software upon power failure, state-of-the-art systems save the program state to non-volatile memory (like FRAM) once energy drops below a critical threshold. This allows the node to resume its specific task exactly where it left off once energy returns, preventing the "reboot-loop" that often renders battery-less devices ineffective during fluctuating power conditions **(Ransford, Lucia, & Levis, p. 2021)**.

## 2.2.8 Emerging Indoor Photovoltaics:

While silicon-based solar is mature, current research in **Perovskite Solar Cells (PSCs)** and **Organic Photovoltaics (OPVs)** is revolutionizing indoor IoT. These materials are optimized specifically for the spectrum of LED and fluorescent lighting, achieving higher efficiency in low-lux environments than traditional outdoor panels. These thin-film, flexible cells allow for the integration of energy harvesting directly into wearable medical sensors and retail "smart labels" that must operate under constant but low-intensity indoor light **(IEEE, 2025)**.

## 2.2.9 Hybrid Energy Management and Machine Learning:

The most recent advancements involve **Hybrid Energy Harvesting**, which simultaneously scavenges from solar, thermal, and RF sources. To manage these complex inputs, researchers are employing **TinyML (on-device Machine Learning)** to predict future energy availability. These intelligent systems dynamically adjust the node's duty cycle increasing sensing frequency during "energy surplus" and entering ultra-low-power sleep during "energy droughts "to achieve a 66% reduction in discovery delay and significantly prolong the network's operational life **(Warden & Situnayake, 2020)**.

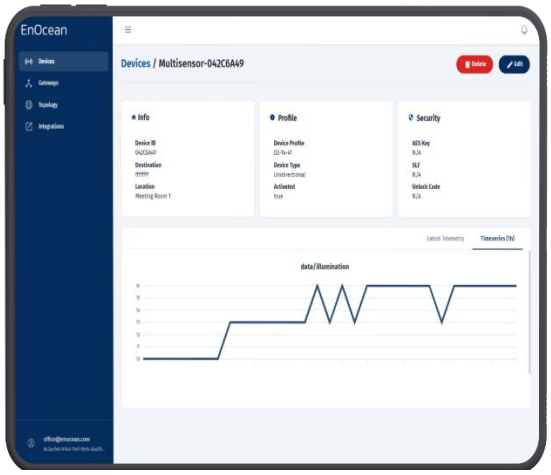
## 2.2.10 Computational RFID and Battery-Free Sensing (The WISP Project):

A unique branch of energy harvesting research focuses on **Computational RFID**, exemplified by the **Wireless Identification and Sensing Platform (WISP)** developed initially by Intel Research and the University of Washington **(Sample, Yeager, Powledge, Mamishew, & Smith, p. 2022)**. Unlike traditional RFID tags that only transmit a static ID, WISP is a fully programmable, battery-free sensing and computing platform. It harvests energy from the radio frequency (RF) energy emitted by standard UHF RFID readers to power a 16-bit microcontroller and a variety of sensors (accelerometers, temperature, and strain gauges). The WISP project is significant because it introduced the concept of **intermittent computing**. Research using this platform addresses the "reboot-loop" problem where a device starts a task, runs out of power, and loses its progress. WISP-related studies have pioneered software techniques that allow a sensor to save its state into non-volatile memory (like FRAM) just before power failure and resume immediately upon the next energy harvest. This work serves as the academic foundation for modern battery-less IoT devices that must perform complex logic in highly transient energy environments **(Sample, Yeager, Powledge, Mamishew, & Smith, 2008)**.

# Previous Systems (or Similar Applications)

This section reviews existing systems and applications that are functionally related to the proposed **multi-source energy-harvesting, battery-less autonomous sensor node**. The reviewed systems provide energy harvesting, sensing, monitoring, or management capabilities similar to the proposed solution.

## 3.1 EnOcean Device Management Platform









- **System name:** EnOcean Device Management Platform
- **URL:** https://www.enocean.com/en/products/self-powered-sensors
- **Platform:** Windows, Linux, macOS via browser
- **Trial license:** Demo kits (vendor-controlled)

## Description:

This platform was developed to configure and monitor EnOcean self-powered sensors used in building automation. It targets building managers and system integrators seeking maintenance-free sensor deployments. While reliable, it employs conservative duty cycles and single-source harvesting, limiting adaptability compared to the proposed system. EnOcean self-powered sensors are commercially deployed devices that harvest energy from indoor light or mechanical motion to enable battery-less operation. They target building automation users but operate with conservative duty cycles **(EnOcean GmbH, 2026)**.

## Limitations:

The EnOcean platform is optimized primarily for ultra-low-power building automation scenarios, where environmental conditions are relatively predictable and energy harvesting typically relies on indoor solar or kinetic sources. While this enables maintenance-free deployments, the architecture assumes constrained workloads and conservative duty cycles to preserve energy stability. The system does not support dynamic integration of multiple heterogeneous energy sources, nor does it provide real-time energy estimation at the firmware level.

## Unresolved Problems:

The platform does not address the coordination of diverse harvesting modalities under fluctuating environmental conditions. It lacks predictive energy modeling, adaptive communication scheduling, and workload prioritization based on instantaneous storage levels. Furthermore, scalability beyond building automation contexts for example, industrial outdoor environments remain insufficiently demonstrated.

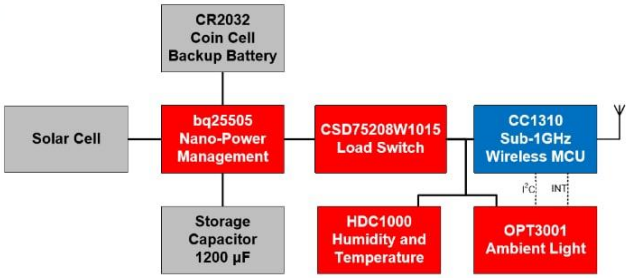
## Need for Current Research:

Future research must extend beyond single-source reliability toward multi-source adaptive energy orchestration. A unified architecture that integrates heterogeneous energy inputs and dynamically modulates sensing and transmission behavior is required. The proposed multi-source autonomous

sensor node addresses these limitations by implementing real-time energy-aware duty cycling and diversified harvesting inputs.

## 3.2 Texas Instruments Energy Harvesting Evaluation GUI (BQ25570)






- **System name:** TI Energy Harvesting Evaluation GUI (part of the BQ25570EVM-206 kit)
- **URL:** https://www.ti.com/product/BQ25570
- **Platform:** Windows
- **Trial license:** Free evaluation software

## Description:

Developed to support evaluation of TI energy-harvesting PMICs, this application visualizes voltage levels and MPPT behavior. It targets embedded system designers and researchers. The software focuses on hardware characterization, such as verifying cold-start voltages and continuous harvesting efficiency down to 100 mV, but it lacks autonomous sensing and system-level intelligence.

## Limitations:

The TI BQ25570 platform is fundamentally a hardware evaluation environment intended to characterize ultra-low-power boost converters and MPPT behavior. While it excels in demonstrating voltage regulation efficiency and cold-start performance, it does not extend into system-level integration. There is no built-in sensing logic, communication stack, or adaptive workload management capability.

## Unresolved Problems:

Although the tool optimizes energy extraction efficiency, it does not solve how harvested energy should be allocated across sensing, processing, and communication tasks. The absence of cross-layer integration between hardware and firmware such as warning microcontrollers of pending power loss via the programmable threshold flags, remains a significant gap for many developers. Additionally, multi-source coordination strategies are not addressed within this single-chip framework.

## Need for Current Research:

Research must transition from component-level efficiency optimization toward holistic autonomous system design. Integrating PMIC regulation with firmware-level energy budgeting and adaptive workload management is necessary for practical battery-less operation. The proposed project expands beyond hardware characterization toward an intelligent energy-driven sensing architecture that utilizes the BQ25570's power-good flags and Nano power buck converter for dynamic system scaling.

## 3.3 Power cast RF Energy Harvesting Monitoring System









- **System name:** Power cast Energy Monitoring Software
- **URL:** https://www.powercastco.com/technologies/rf-power
- **Platform:** Web-based / Windows
- **Trial license:** Hardware evaluation kits (time-limited)

## Description:

This system monitors RF-powered IoT devices in wireless power environments. It targets engineers deploying RF-powered sensors. Dependence on dedicated RF transmitters and lack of hybrid harvesting limit its flexibility. Their technology uses a "Power caster" transmitter that broadcasts radio waves (usually in the 915MHz ISM band). The receiving end uses a high-efficiency **rectjenna** (rectifying antenna) to convert those radio waves back into DC electricity. This is particularly useful in industrial settings where sensors are buried in walls or located in rotating machinery were changing batteries or using solar is impossible **(Powercast Corporation, 2025)**.

## Limitations:

Power cast systems demonstrate effective RF-based wireless power transfer; however, practical deployment typically requires dedicated RF transmitters to achieve usable power levels. Ambient RF energy density in uncontrolled environments remains insufficient for sustained sensing and wireless communication.

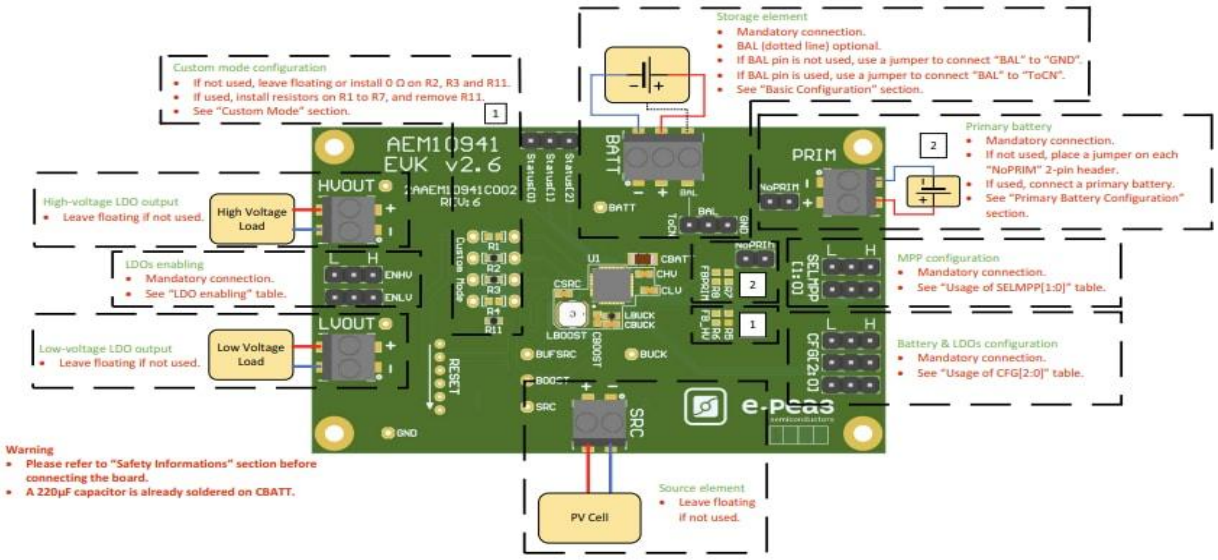
## Unresolved Problems:

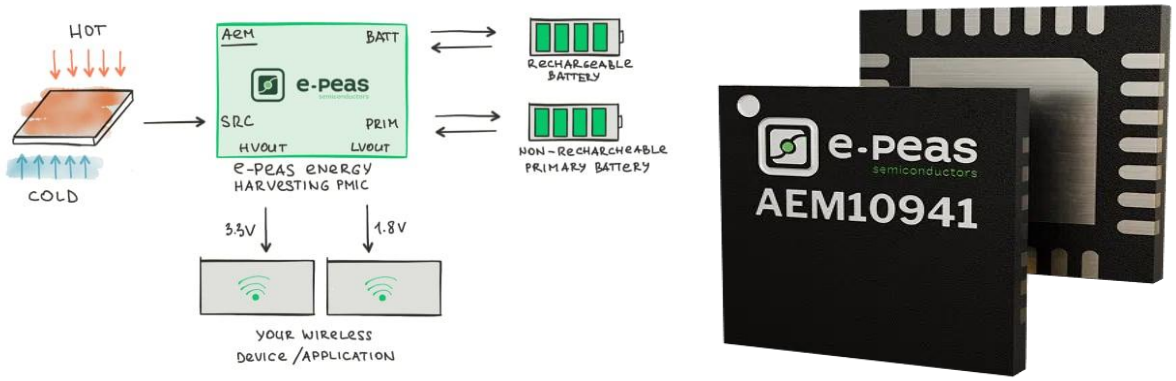
Reliance on infrastructure-based RF transmitters limits scalability and cost-efficiency. Additionally, the architecture does not integrate alternative energy sources to mitigate RF intermittency or low ambient power levels. Energy storage management under fluctuating RF conditions is also minimally addressed.

## Need for Current Research:

Hybrid architectures that treat RF harvesting as a supplementary source within a broader multi-source ecosystem are required. Research must investigate coordinated harvesting where RF energy complements solar and magnetic inputs to improve resilience. The proposed system directly responds to this need by incorporating RF harvesting as one component of a diversified energy strategy.

# 3.4 E-peas Energy Harvesting Demonstration Software (AEM10941)




- **System name:** e-peas Energy Harvesting Demo Software
- **URL:** [https://e-peas.com/product/aem10941](https://e-peas.com/product/aem10941)
- **Platform:** Windows
- **Trial license:** Evaluation kits

## Description:

Designed to demonstrate photovoltaic harvesting with MPPT and super capacitor storage, this software targets hardware engineers. It is hardware-centric and does not implement predictive or adaptive task scheduling. The **AEM10941** is often the top choice for photovoltaic (PV) harvesting because of its ultra-low power footprint. It can start harvesting energy when the input voltage is as low as 380mV, which is critical for indoor solar panels that may only produce a tiny amount of voltage. It manages the entire energy chain: it extracts power from the PV cell, stores it in a battery or super capacitor, and provides two regulated voltages to power an application's MCU and radio (e-peas, 2024).

## Limitations:

The e-peas AEM10941 platform provides high-efficiency photovoltaic harvesting with MPPT and super capacitor compatibility. However, it remains hardware-centric, focusing on energy conversion performance rather than autonomous workload adaptation.

## Unresolved Problems:

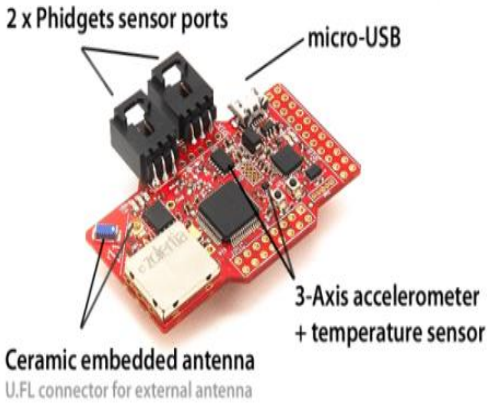
There is limited integration between harvested energy metrics and real-time system behavior adjustment. The system does not incorporate predictive duty cycling or adaptive communication strategies based on stored energy thresholds.

## Need for Current Research:

Future designs must merge efficient PMIC technology with intelligent firmware capable of adjusting sensing intervals, transmission frequency, and sleep states dynamically. The proposed system advances this direction by embedding adaptive duty-cycling algorithms into the harvesting architecture.

## 3.5 Zolertia IoT Energy-Harvesting Node (Zolertia IPv6/6LoWPAN Ubidots client)




- **System name:** Zolertia IPv6/6LoWPAN Ubidots client
- **URL:** https://www.hackster.io/4348/zolertia-ipv6-6lowpan-ubidots-bcde84
- **Platform:** Web-based
- **Trial license:** Open-source / research use

## Description:

This dashboard supports monitoring of low-power IoT nodes in research deployments. Target users include researchers and universities. Most nodes remain battery-assisted, limiting full autonomy. Zolertia platforms, such as the Z1 mote used in the referenced IPv6/6LoWPAN project, are engineered for the Industrial Internet of Things (IoT). They employ a Texas Instruments CC2538 SoC with dual-radio support for flexible 2.4GHz and sub-GHz communication, forming resilient mesh networks. A core design philosophy is its "harvesting-ready" architecture: the hardware includes dedicated headers and low-quiescent-current components, allowing for the straightforward integration of external energy-harvesting modules for example solar, wind, or thermal **(Lignan, Antonio, p. 2022)**.

## Limitations:

Although Zolertia platforms support low-power IoT experimentation, many deployments remain battery-assisted. Energy harvesting capabilities are not consistently integrated into adaptive scheduling frameworks.

## Unresolved Problems:

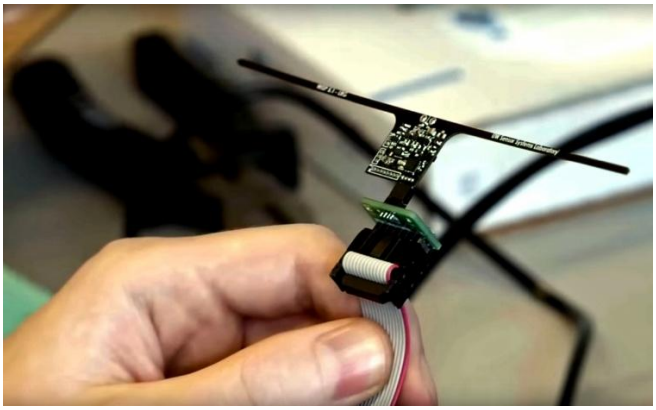
Energy-neutral operation without battery backup remains insufficiently demonstrated. Real-time energy estimation and dynamic workload prioritization are not fully integrated into the monitoring architecture.

## Need for Current Research:

A truly battery-less system requires multi-source harvesting combined with firmware-level decision-making that eliminates fallback battery dependency. The proposed research addresses this by designing a fully autonomous node without reliance on auxiliary battery support.

## 3.6 Wireless Identification and Sensing Platform (WISP) Tools





- **System name:** WISP Monitoring and Debug Tools
- **URL:** https://sites.google.com/uw.edu/wisp-wiki/home
- **Platform:** Windows, Linux and MacOS (PC-based research tools)
- **Trial license:** Open research platform

## Description:

Developed to demonstrate battery-free RF-powered sensing, WISP targets academic researchers. Operation is constrained by reader proximity and intermittent power.

## Limitations:

The **Wireless Identification and Sensing Platform (WISP)** is an open source and fully hackable battery-free platform for experimentation with low-power sensing, computation, and communication **(Smith, 2021)**. WISP technology depends heavily on RFID reader proximity for energy harvesting. The intermittent nature of RF supply causes unstable sensing cycles and limits operational continuity.

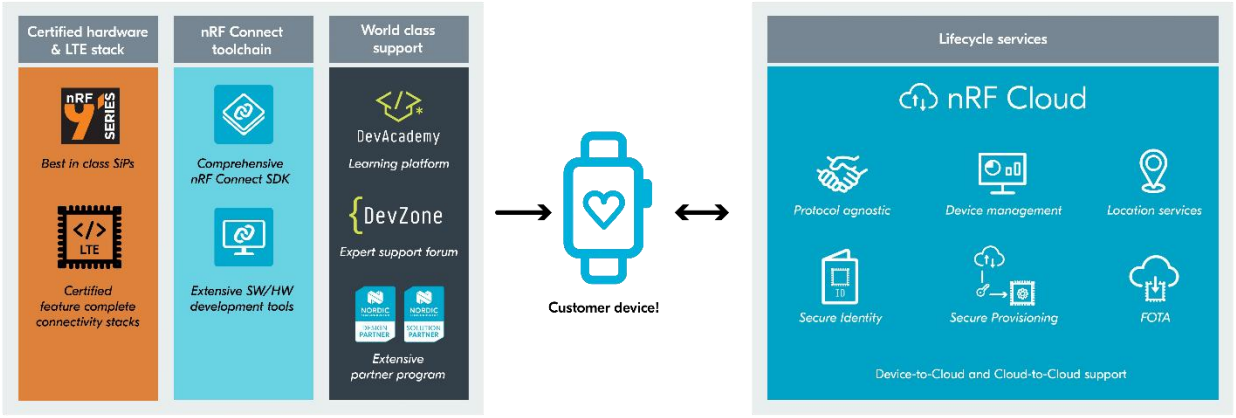
## Unresolved Problems:

Continuous environmental monitoring is impractical due to inconsistent power availability. The system lacks integration with other ambient sources to stabilize operation under varying environmental conditions.

## Need for Current Research:

A multi-source architecture that supplements RF harvesting with solar and magnetic energy can mitigate intermittency and improve reliability. The proposed system directly expands on WISP's concept by removing single-source dependency.

## 3.7 Nordic Semiconductor BLE Energy Profiling Tools




- **System name:** Nordic Power Profiler
- **URL:** https://www.nordicsemi.com/Products/Wireless/Low-power-cellular-IoT
- **Platform:** Windows
- **Trial license:** Free with hardware

## Description:

The Nordic Power Profiler is a dedicated analysis tool designed for embedded developers to measure and evaluate the power consumption of Bluetooth Low Energy (BLE) devices. It focuses on quantifying how much energy specific operations especially wireless communications use in real application scenarios. While the tool offers detailed insight into where and how energy is consumed, it does not include features for energy harvesting or autonomous power management; rather, it highlights that communication operations often dominate overall energy use.

## Limitations:

The Nordic Power Profiler functions only as a measurement and profiling instrument; it does not provide any mechanisms for harvesting energy from the environment. It does not include adaptive workload control or dynamic power management strategies that react to available energy.

## Unresolved Problems:

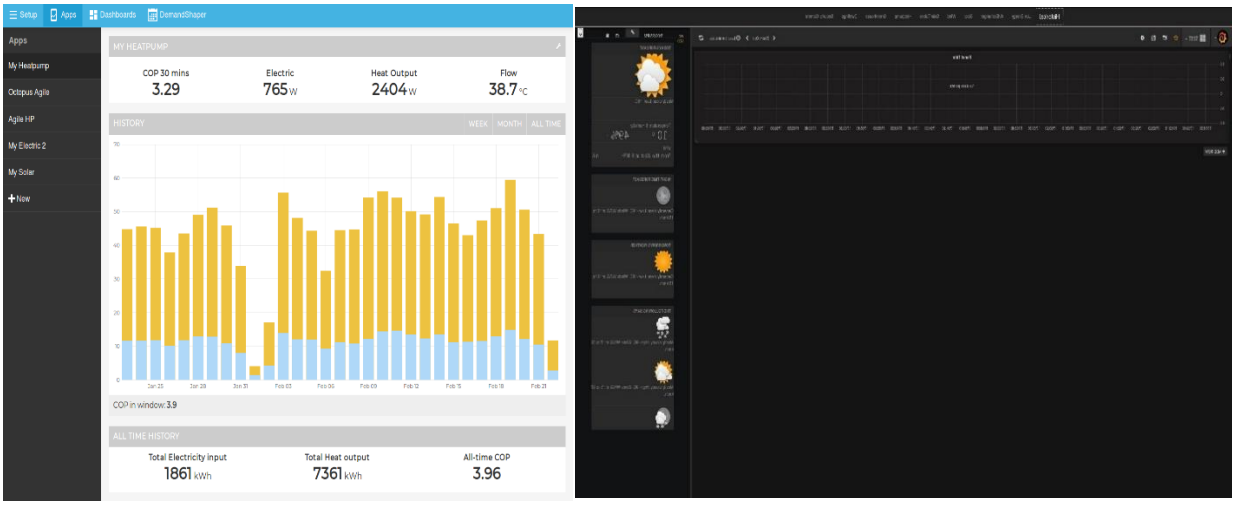
Although the Profiler can identify high-consumption elements such as BLE radio activity it does not offer integrated methods to *respond* to those findings under constrained energy conditions. For example:

- It cannot automatically reduce transmission frequency when battery levels are low of which a multi-energy sources would sustain the system.
- It lacks native support for shaping device workflows based on consumption patterns

## Need for Current Research:

Research must integrate consumption profiling insights with adaptive firmware strategies to dynamically reduce communication overhead when harvested energy is insufficient. The proposed system incorporates energy-aware scheduling mechanisms inspired by such profiling insights.

## 3.8 Open Energy Monitor IoT Monitoring Platform




- **System name:** Open Energy Monitor
- **URL:** https://openenergymonitor.org
- **Platform:** Web-based
- **Trial license:** Open-source

## Description:

Originally developed for energy monitoring, this platform supports sensor data visualization. It targets hobbyists and researchers. Battery-less harvesting is not native, limiting suitability for autonomous sensing.

## Limitations:

The platform focuses on energy data visualization rather than autonomous energy management. It assumes stable and continuous power supply.

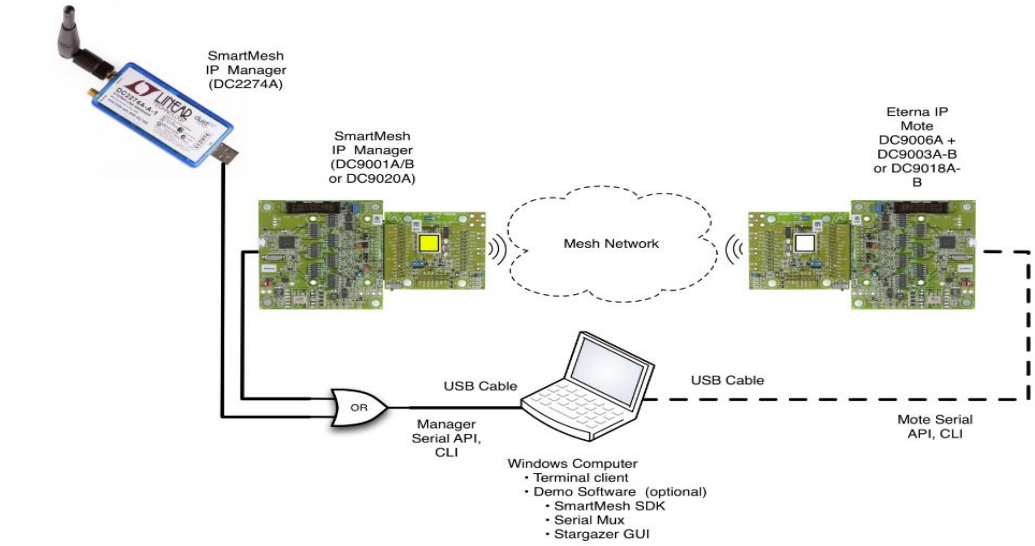
## Unresolved Problems:

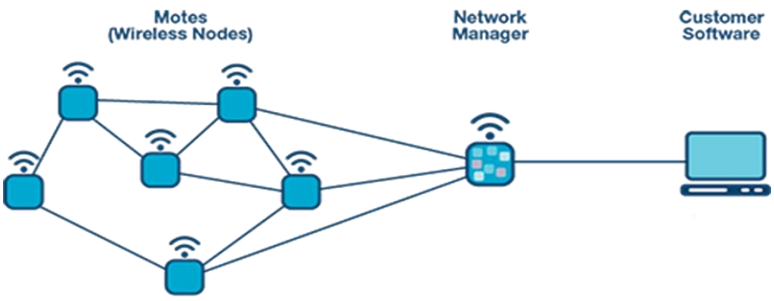
There is no mechanism to adapt sensing frequency or communication intervals based on available harvested energy.

## Need for Current Research:

Future IoT platforms must integrate energy-awareness at the application layer. The proposed research contributes by embedding energy-level-based workload modulation within the sensing node itself.

## 3.9 Smart Mesh IP Power Monitoring Tools




- **System name:** Smart Mesh Network Manager
- **URL:** https://www.analog.com/media/en/technical-documentation/user-guides/smartmesh_ip_tools_guide.pdf
- **Platform:** Web-based / PC
- **Trial license:** Evaluation-based

## Description:

This system manages ultra-low-power wireless sensor networks. It targets industrial IoT users. Nodes typically rely on batteries rather than ambient harvesting **(Analog Devices, 2021)**.

## Limitations:

Existing solutions often lack integrated Maximum Power Point Tracking (MPPT) at the network protocol level. While chips like the e-peas AEM10941 support hardware-level MPPT ratio configuration for PV cells, this logic is not typically communicated to the mesh routing layer. Additionally, batteries remain finite and create maintenance cycles that energy harvesting aims to eliminate.

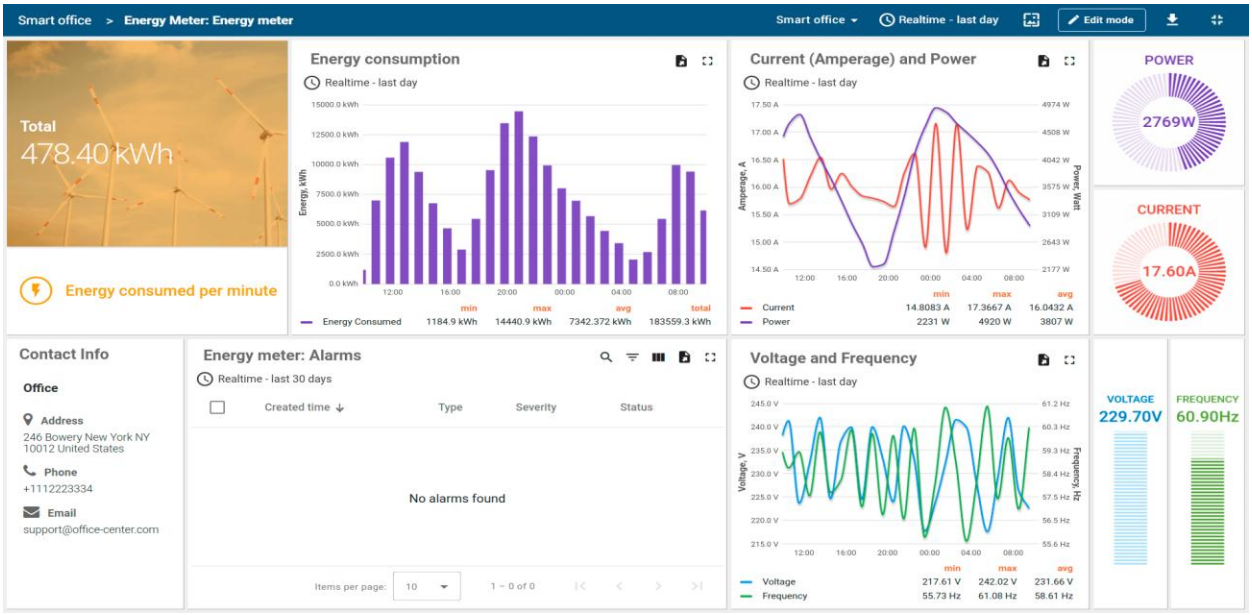
## Unresolved Problems:

Energy-neutral mesh operation under fluctuating harvested power remains insufficiently explored. A major challenge is maintaining a continuous power rail for high-demand "burst" operations, such as wireless data transmission, when harvested energy is minimal. Ensuring **cold-start capability** from extremely low voltages remains a bottleneck for starting up a mesh network after a period of total darkness or thermal inactivity.

## Need for Current Research:

There is a critical need to research **adaptive monitoring** that adjusts reporting frequency based on the current state of a node's energy buffer for example, super capacitors or thin-film batteries). Research into **autonomous power management** like that found in EnOcean's multi-sensors which combine solar harvesting with temperature and humidity sensing is essential for creating a "digital twin" of industrial environments that requires zero maintenance.

## 3.10 Things Board IoT Monitoring Platform




- **System name:** Things Board
- **URL:** [https://thingsboard.io](https://thingsboard.io)
- **Platform:** Web-based
- **Trial license:** Community Edition (free)

## Description:

Things Board is a generic IoT monitoring platform used to visualize sensor data. It targets IoT developers and enterprises. Energy harvesting awareness and adaptive duty cycling are not natively supported.

## Limitations:

Things Board functions as a telemetry visualization and rule engine platform without intrinsic energy-awareness mechanisms.

## Unresolved Problems:

The platform assumes continuous device uptime and stable connectivity, which is incompatible with intermittent energy harvesting conditions.

## Need for Current Research:

IoT management platforms must evolve toward energy-adaptive frameworks that support variable communication intervals. The proposed research demonstrates how energy-aware node behavior can complement such platforms.

## 4. Lessons Learnt from the Review

The literature as reviewed or rather highlighted several key lessons on the systems. First, reliance on a single ambient energy source is insufficient for reliable long-term operation due to environmental variability. Multi-source energy harvesting significantly improves robustness by compensating for the absence or degradation of individual sources.

Second, hardware-focused evaluation tools alone are inadequate for autonomous operation without intelligent firmware capable of energy estimation and adaptive control.

Third, RF energy harvesting is most effective as a supplementary source rather than a primary power supply.

Fourth, super capacitor-based storage offers clear advantages for battery-less systems due to long cycle life and fast charging.

Finally, adaptive duty cycling is essential for aligning energy consumption with harvested energy and preventing system brownout.

## 5. A Critique of the Review (Gaps, Inconsistencies, Omissions)

Despite extensive research and numerous existing systems, several gaps remain unresolved. Many prior works focus on isolated components. Such as harvesters, power management circuits, or monitoring dashboards rather than integrated autonomous sensor systems. Single-source dependence remains common, even though environmental variability is well known. Furthermore, most IoT platforms lack energy-awareness and assume stable power availability, making them unsuitable for battery-less operation.

Another major omission is the limited integration of energy harvesting with adaptive firmware-level decision-making. While MPPT and storage are widely addressed, fewer systems incorporate intelligent duty cycling based on real-time energy availability. Additionally, many studies evaluate performance under laboratory conditions without addressing long-term deployment variability. These gaps collectively justify the need for a unified multi-source energy-harvesting system that integrates power management, energy storage, and adaptive control to achieve reliable autonomous operation.

## 2.16 Chapter Summary

This chapter critically reviewed existing academic research, commercial energy-harvesting platforms, and current engineering practices relevant to autonomous sensor systems. It examined solar, magnetic, and radio-frequency energy harvesting techniques, compared energy storage technologies, and discussed the role of adaptive energy management in battery-less embedded systems. A comparative analysis of existing platforms revealed that most current solutions focus on single-source harvesting, limited adaptive intelligence, or battery-assisted operation. From these observations, the chapter identified clear research gaps that justify the proposed hybrid architecture integrating multiple harvesting sources, super capacitor-based storage, and adaptive duty-cycling firmware.

The findings of this literature review provide the theoretical and practical foundation for the engineering decisions presented in **Chapter Three**, where the complete system analysis, architecture, hardware design, software design, mathematical modelling, and engineering calculations of the proposed Multi-Source Energy-Harvesting Autonomous Sensor Node will be presented in detail.

# CHAPTER THREE:

# SYSTEM DESIGN

## 3.1. Introduction

This chapter presents the complete system design of the Multi-Source Energy-Harvesting Autonomous Sensor Node. The system is designed to operate autonomously using ambient energy sources while adapting its workload based on available energy to ensure implementation stability and reduce unnecessary complexity, the system adopts a simplified communication architecture. Instead of wireless or cloud-based (IoT) communication, telemetry data is transmitted directly via USB serial to a laptop for logging and analysis This approach ensures:

1. Minimal system complexity
2. Reduced firmware overhead
3. Deterministic communication
4. Stable experimental evaluation
5. Strong focus on adaptive energy management

## 3.2. System Objectives

The system is designed to:

1. Harvest energy from multiple ambient sources namely; solar, magnetic, and radio frequency.
2. Store harvested energy in super capacitors.
3. Monitor energy storage levels, adapt sensing and operational behavior based on energy availability, transmit telemetry data for evaluation by use of scheduled algorithms.

## 3.3. RESEARCH METHODOLOGY

## Iterative Engineering Process Model

The research methodology model employs a design based and experimental approach, focusing on the development of a functional prototype and performance evaluation. The study follows an engineering design process consisting of system analysis, design, implementation, and testing.

Initially, a comprehensive literature review was conducted to understand existing energy harvesting technologies, power management strategies, and intelligent duty-cycling mechanisms. Based on the identified research gaps, system requirements are defined, followed by the design of the overall architecture and subsystem models. Hardware design is carried out using electronic design principles.

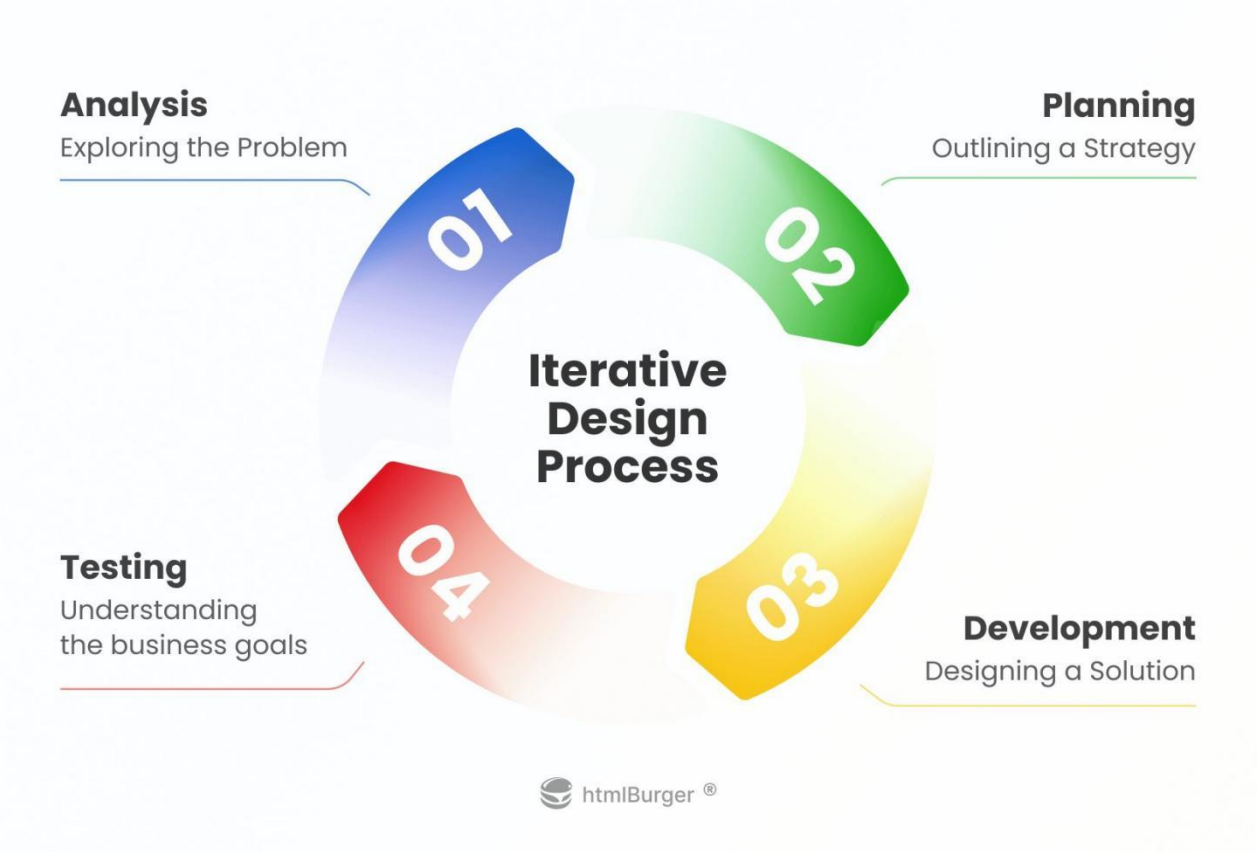



**Figure 3:** Iterative Engineering Process Model Diagram

# METHODOLOGY STAGES

## 3.2.1 Stage One: Requirement Analysis

Extensive review of existing academic and industrial energy harvesting systems was conducted to identify design gaps. Functional and non-functional requirements were derived from this review.

### Functional Requirements:

#### System Requirements

The proposed system shall:

1. harvest ambient energy from solar, magnetic, and radio-frequency sources.
2. convert and regulate harvested energy using a power management unit.
3. store harvested energy in a super capacitor-based storage unit.
4. monitor energy levels, estimate available energy, adaptive sensing and communication duty cycles according to the available energy.
5. transmit sensor data wirelessly using a low-power communication module.
6. enter low-power or sleep modes when available energy falls below the required operating threshold.

### Non-Functional Requirements

The proposed system shall:

1. **Input energy from multiple ambient sources** (e.g., solar, magnetic, and RF harvesting modules) through integrated hardware interfaces designed to minimize overall power consumption.
2. **Process incoming harvested energy using power-management circuitry and embedded control algorithms** to regulate voltage levels and distribute power efficiently to system components.
3. **Output regulated electrical power to sensor nodes and communication modules**, ensuring stable and reliable operation.

## 3.2.2 Stage Two: System Architecture Design and Modeling

In the second stage, the overall system architecture of the proposed multi-source energy-harvesting autonomous sensor node is being designed. This stage is involving the definition of interactions among the energy harvesting units (solar, magnetic, and radio frequency), the power management subsystem, energy storage elements, the microcontroller unit, sensors, and the wireless communication module. System-level modeling is being performed to analyze energy flow, power conversion efficiency, and operational feasibility under varying ambient energy conditions. The architectural design is ensuring modularity to allow independent optimization of individual subsystems.

**The system consists of four major subsystems:**

1. Energy Harvesting Subsystem
2. Power Management and Storage Subsystem
3. Processing and Sensing Subsystem
4. Serial Communication and Data Logging Subsystem

## 3.2.2 Overview System Architecture

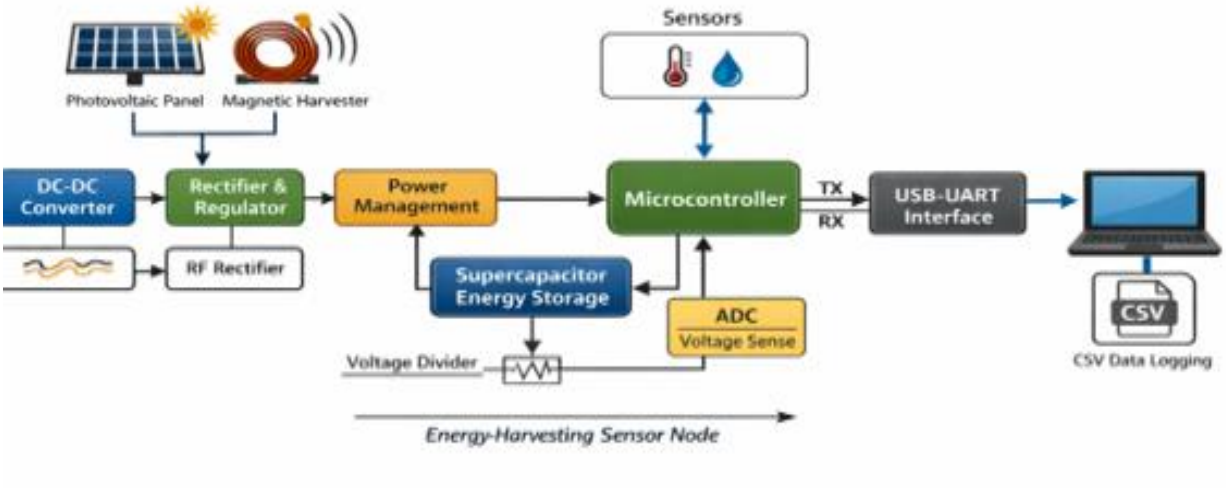



**Figure 3.2.2: Overview High-Level System Block Diagram (a)**

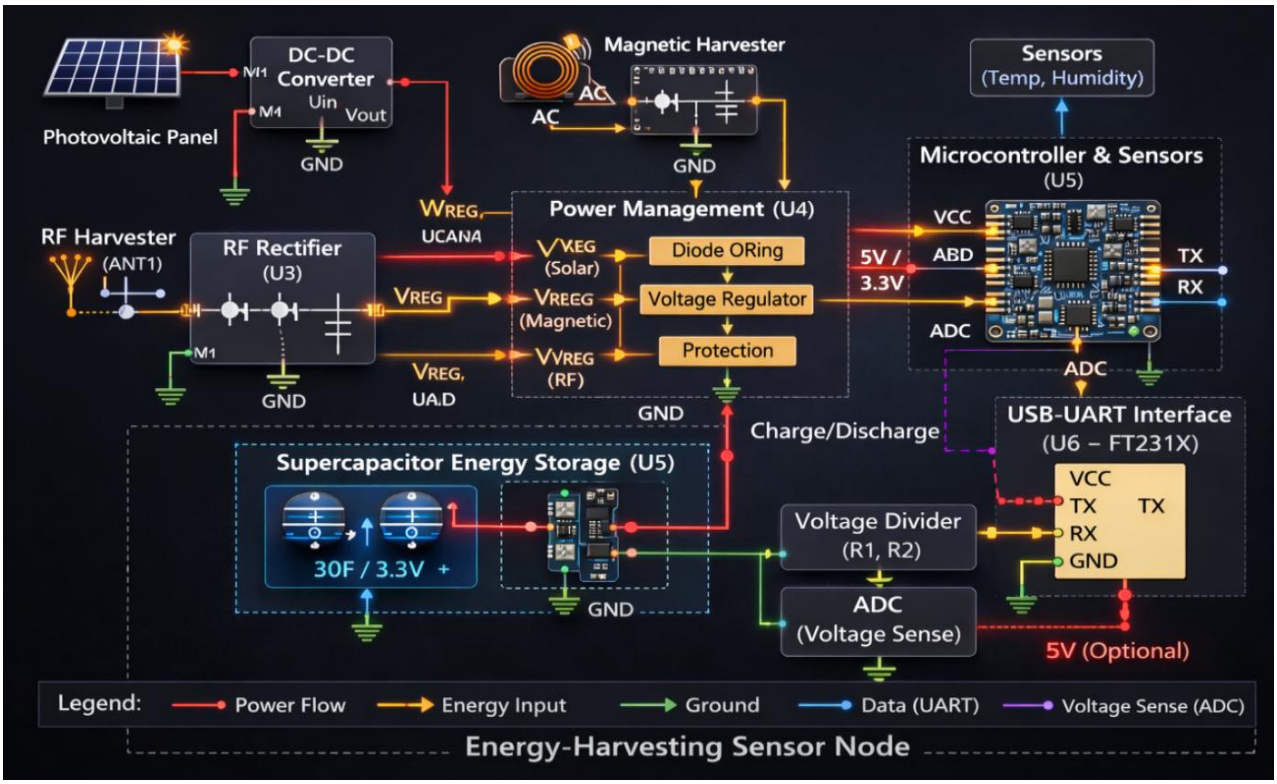



**Figure 3.2.2: Detailed High-Level Circuit System Block Diagram (b)**

## 3.2.3 Stage Three: Hardware Design and Component Selection

During this stage, detailed hardware design and component selection are being carried out. Electronic components are being selected based on ultra-low-power characteristics, efficiency, availability, and suitability for energy harvesting applications. Circuit schematics are being developed to integrate the energy harvesting sources, PMIC, super capacitor-based energy storage, microcontroller, sensors, and communication modules. Following schematic verification, the hardware design is being implemented through printed circuit board (PCB) design, ensuring compact layout, reduced power losses, and reliable electrical connectivity.

## 3.2.3.1 Electrical Design Specifications

To ensure a practical and implementable design, the following electrical parameters are defined:

| **Parameter** | **Value** | **Justification** |
|---|---|---|
| Solar Panel Output | 5V (nominal), 200 mA (max) | Suitable small PV module |
| Magnetic Harvester Output | 2–8 V AC (variable) | Depends on field strength |
| RF Harvester Output | 0.2–1.5 V DC (low power) | Ambient RF dependent |
| Regulated System Voltage | 3.3 V ±5% | Standard MCU operating voltage |
| Super capacitor Rating | 30 F, 3.0 V | Energy buffer |
| ADC Reference Voltage | 3.3 V | Matches regulated supply |
| UART Baud Rate | 9600 bps | Stable, low overhead |

The regulated 3.3 V supply ensures compatibility with ultra-low-power microcontrollers and digital sensors.

# 3.2.3.2 Energy Harvesting Subsystem

The system integrates multiple ambient energy sources to increase reliability under varying environmental conditions**. (Gloomier Solar Power Station, 2022; Touwsrivier CPV Solar Project, 2025; Nzeanorue, 2025).**

# 3.2.3.2.1 Solar Harvesting Section

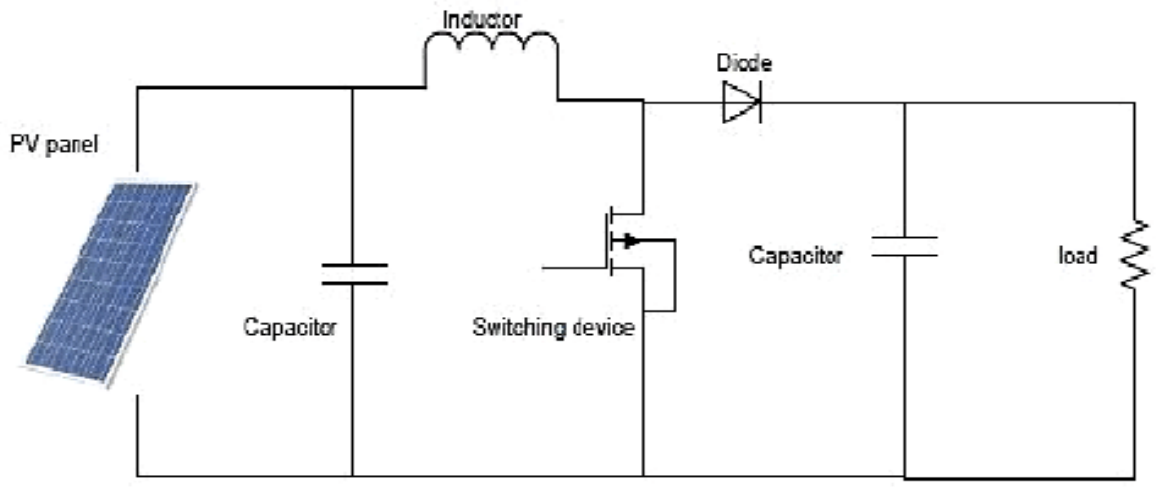



**FIGURE 3.2.3.2.1: Solar Harvesting Section**

### Photovoltaic Panel

Converts sunlight into DC electrical energy. Provides variable voltage depending on light intensity.

### DC-DC Converter

Steps up or regulates the solar voltage, ensures stable intermediate DC output and prevents under voltage operation of the system.

**Role:** Makes solar power usable for charging and regulation.

# 3.2.3.2.2 Magnetic Harvesting Section

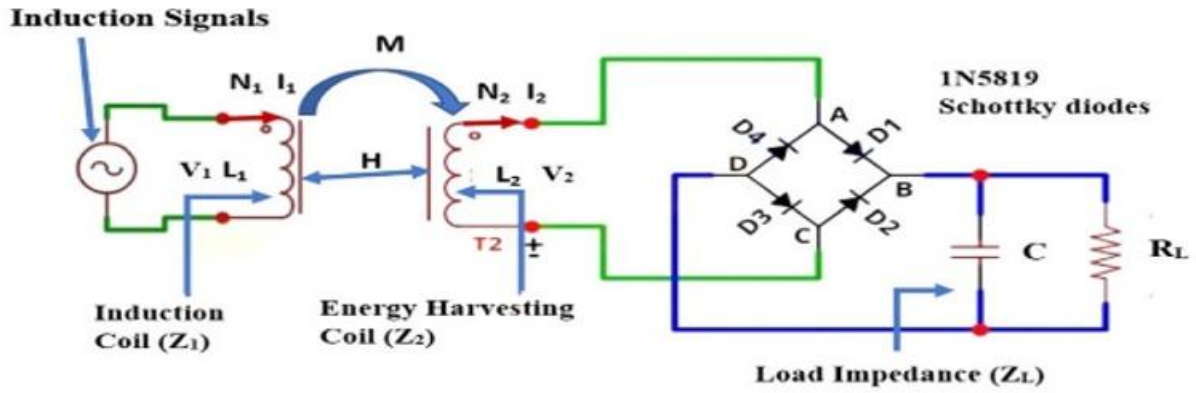



**FIGURE 3.2.3.2.2: Magnetic Harvesting Section (a)**

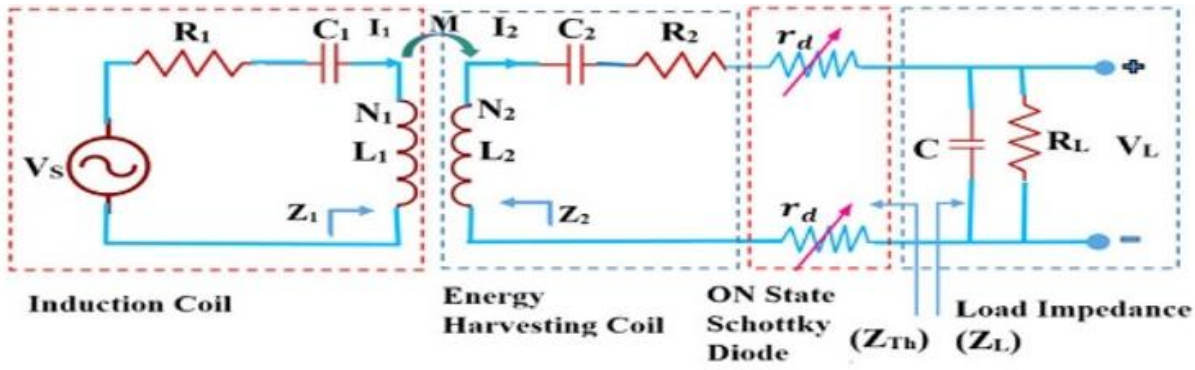



**FIGURE 3.2.3.2.2: Magnetic Harvesting Section (b)**

### Magnetic Harvester (Coil + Core)

Converts changing magnetic fields into AC voltage (Faraday's Law), Output is typically low-level AC.

### Rectifier & Filtering Network

Converts AC to DC, Smoothing ripple voltage.

**Role:** Converts ambient magnetic energy into usable DC power.

# 3.2.3.2.3 RF Harvesting Section

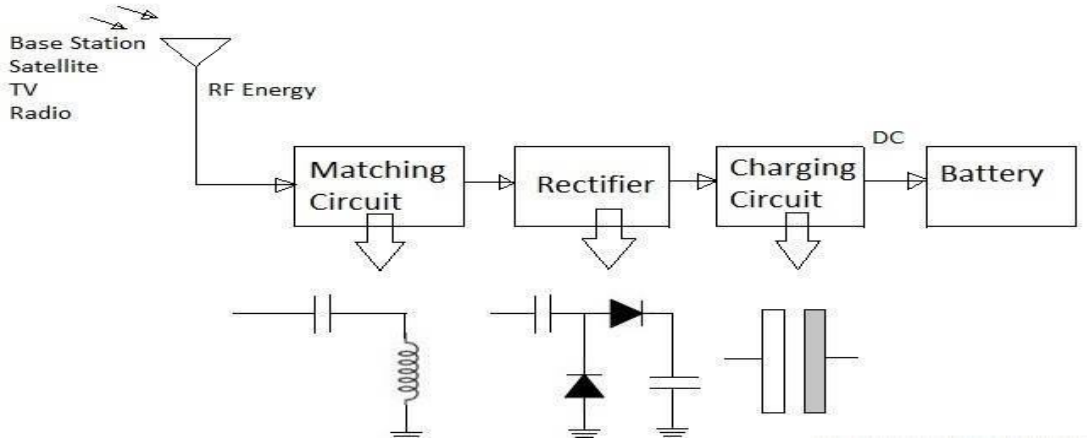



**FIGURE: 3.2.3.2.3 RF Harvesting Section**

### RF Antenna & RF Rectifier Circuit

Captures ambient RF energy (radio waves).

Converts RF AC signals into DC, typically uses diode-based rectification.

### Filter Capacitors

Smoothens the rectified signal.

**Role:** Harvests low-power RF signals and converts them into usable DC energy.

### The harvesting paths include:

Solar Panel →Blocking Diode → DC-DC Converter → Storage Bus

Inductive Coil → Bridge Rectifier → Filter Capacitor → Regulator → Storage Bus

RF Antenna → Matching Network → RF Rectifier → Filter → Storage Bus All energy sources converge at a regulated storage node.

# 3.2.3.2.4 Power Management and Storage Subsystem

This subsystem ensures safe storage and regulation of harvested energy **(Kumar, 2023).**

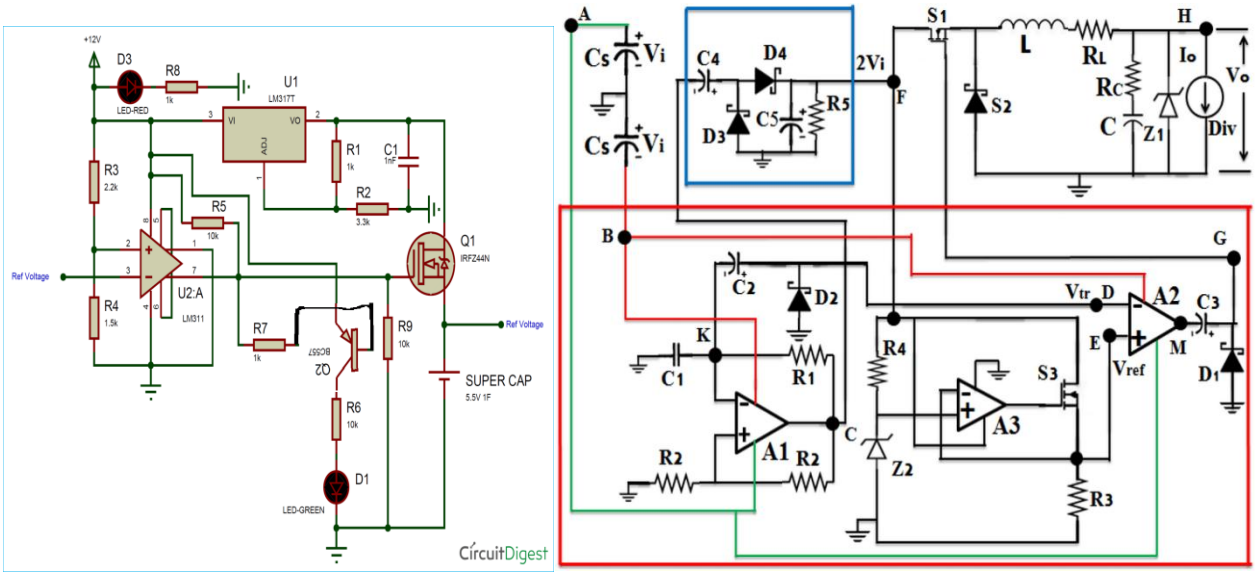



**Figure 3.2.3.2.4: Energy Storage Circuit**

At this stage there is energy rectification, voltage regulation use of super capacitor to bank voltage and a is divider connected to MCU ADC

The super capacitor is selected due to:

1. High cycle life
2. Fast charge capability
3. Suitability for intermittent power
4. Reduced maintenance compared to batteries

The capacitor voltage is continuously monitored to determine energy state.

# 3.2.3.2.5 Central Power Management Unit

### Function:

Combines outputs from solar, magnetic, and RF harvesters, regulates voltage (VREG), prevents reverse current flow and protects against overvoltage.

**Role:** Acts as the energy traffic controller of the entire system.

# 3.2.3.2.5.1 Super capacitor Energy Storage

Stores harvested energy, provides burst current capability for microcontroller operation and handles fluctuations from intermittent harvesting sources.

**Role:** Acts as the system battery (but faster charge/discharge).

# 3.2.3.2.5.2 Voltage Divider + ADC Monitoring

### Voltage Divider

Scales down super capacitor voltage to safe ADC levels.

### ADC (inside Microcontroller)

Measures stored energy voltage, enables energy-aware operation (sleep, wake, duty cycling).

**Role:** Allows intelligent energy management.

# 3.2.3.2.6 Energy Storage Calculation

# Super capacitor Energy Analysis

The energy stored in a capacitor is calculated using the formula:

$$\mathbf{E = \frac{1}{2} cv^2}$$

Where:
C = 30 F

V = 3.0 V

$$E = 0.5 \times 30 \times (3)^2$$

E = **135 Joules**

If the system consumes 30 mW average:

$$\text{Runtime} = \frac{\text{Energy}}{\text{Power}}$$

$$\text{Runtime} = \frac{135}{0.03} = 4500 \text{ seconds}$$

Runtime = **75 minutes**

This demonstrates that the system can continue operating for over one hour without energy input, improving resilience to intermittent harvesting **(Kularatna, 2023).**

# 3.2.3.2.7 Processing and Sensing Subsystem

This subsystem performs sensing, adaptive control, and communication.

**Components:** Ultra-low-power microcontroller, environmental sensors (e.g., temperature, humidity), ADC channel for voltage monitoring and a USB interface for serial transmission.

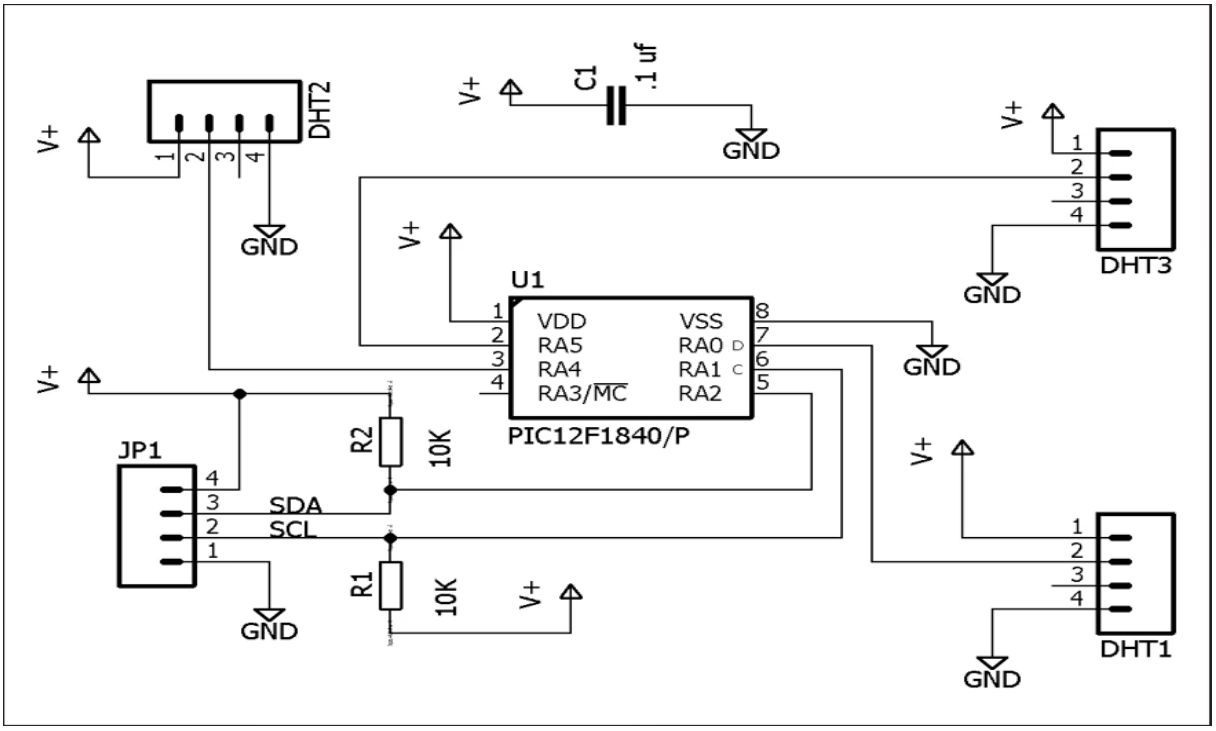



**Figure 3.2.3.2.7: MCU and Sensor Interfacing**

Connections include:

1. I2C for digital sensors
2. ADC for capacitor voltage measurement
3. GPIO for control signals
4. Regulated Voltage current supply from storage

## 3.2.3.2.7.1 Microcontroller & Sensors

**Microcontroller:** It's the core processing unit that reads sensor data, monitors stored energy, controls system operation and manages UART communication.

May communicate via I²C, SPI, or analog input and Powered by regulated 3.3V or 5V from power management.

**Role:** This is the brain of the system.

**Sensors:**

1. Temperature sensor.
2. Humidity or environmental sensor.

# Power Budget Analysis

$$\mathbf{p = v \times i}$$

$$\boldsymbol{p = 3.3 \times 0.0195 \approx 64mW}$$

If active time = 1 second every 10 seconds average current:

$$\boldsymbol{I_{avg} = \frac{(19.5 \times 1) + (0.005 \times 9)}{10}}$$

$$\mathbf{I_{avg=2mA}}$$

$$\mathbf{P_{avg=6.6mW}}$$

Estimated component current consumption:

| Component | Active Current | Sleep Current |
|---|---|---|
| MCU | 10 mA | 5 µA |
| Temperature Sensor | 1.5 mA | 2 µA |
| USB-UART | 8 mA | 1 mA |
| Total (Active) | ~19.5 mA | — |

**At 3.3 V:** This demonstrates that duty cycling significantly reduces energy consumption **(Kansal et al., 2007).**

# 3.2.3.2.8 Serial Communication and Data Logging Subsystem

To minimize complexity, the system uses USB serial communication instead of wireless transmission.

**Signal flow:** Microcontroller UART → USB-UART Converter → USB Cable → Laptop → Python Logger → CSV File

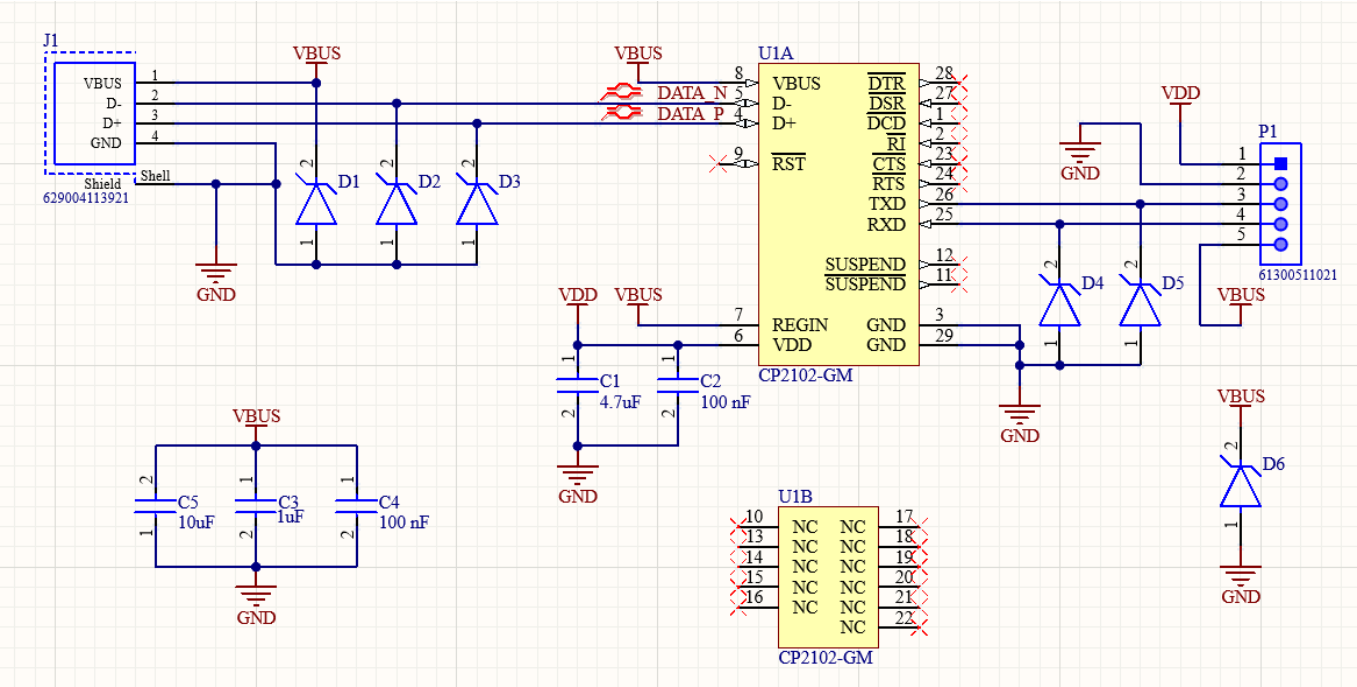



**FIGURE: 3.2.3.2.8 Serial Communication and Data Logging Subsystem**

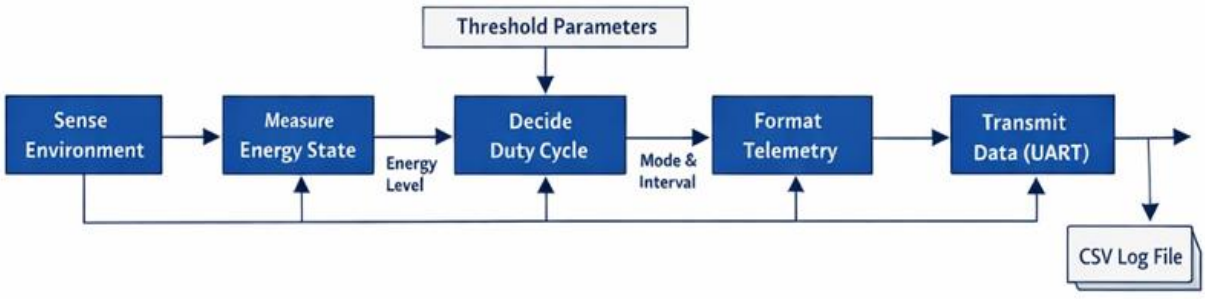



**Figure 3.2.3.2.8: Data flow Diagram**

## 3.2.3.2.8 USB-UART INTERFACE

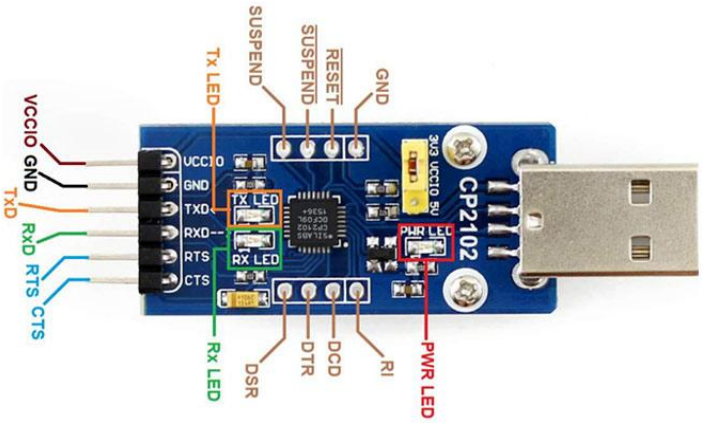



**Figure 3.2.3.2.8: USB Serial Interface**

Converts MCU UART (TX/RX) signals to USB protocol then allows connection to the laptop in order to enable CSV logging and If USB was removed Data may instead be sent via direct UART pins.

# Overall Energy & Signal Flow

### Power Path:

1. Solar / Magnetic / RF
   → Rectification
   → Regulation
   → Super capacitor
   → Voltage Regulator
   → Microcontroller + Sensors

### Data Path:

2. Sensors
   → Microcontroller
   → UART (TX/RX)
   → Logging system

## 3.2.3.2.8.1 System Behavior Summary

| Component | Primary Role |
|---|---|
| Solar Panel | Main energy source |
| Magnetic Harvester | Secondary energy source |
| RF Harvester | Backup ultra-low energy source |
| Rectifier | AC → DC conversion |
| Power Management | Voltage control & merging |
| Super capacitor | Energy storage |
| Voltage Divider | Safe voltage sensing |
| ADC | Energy monitoring |
| Microcontroller | Processing & control |
| Sensors | Environmental data collection |
| UART | Data communication |

## 3.2.4 Stage Four: Firmware Development and Energy-Aware Control Algorithms

At this stage, firmware for the autonomous sensor node is being developed. Embedded software is being designed to monitor energy availability, manage power states, and execute adaptive duty-cycling algorithms. The firmware is dynamically adjusting sensing, processing, and communication operations based on energy levels stored in the super capacitor. Energy-aware scheduling techniques are being implemented to minimize power consumption, prevent brown-out conditions, and maintain stable system operation under fluctuating ambient energy supply.

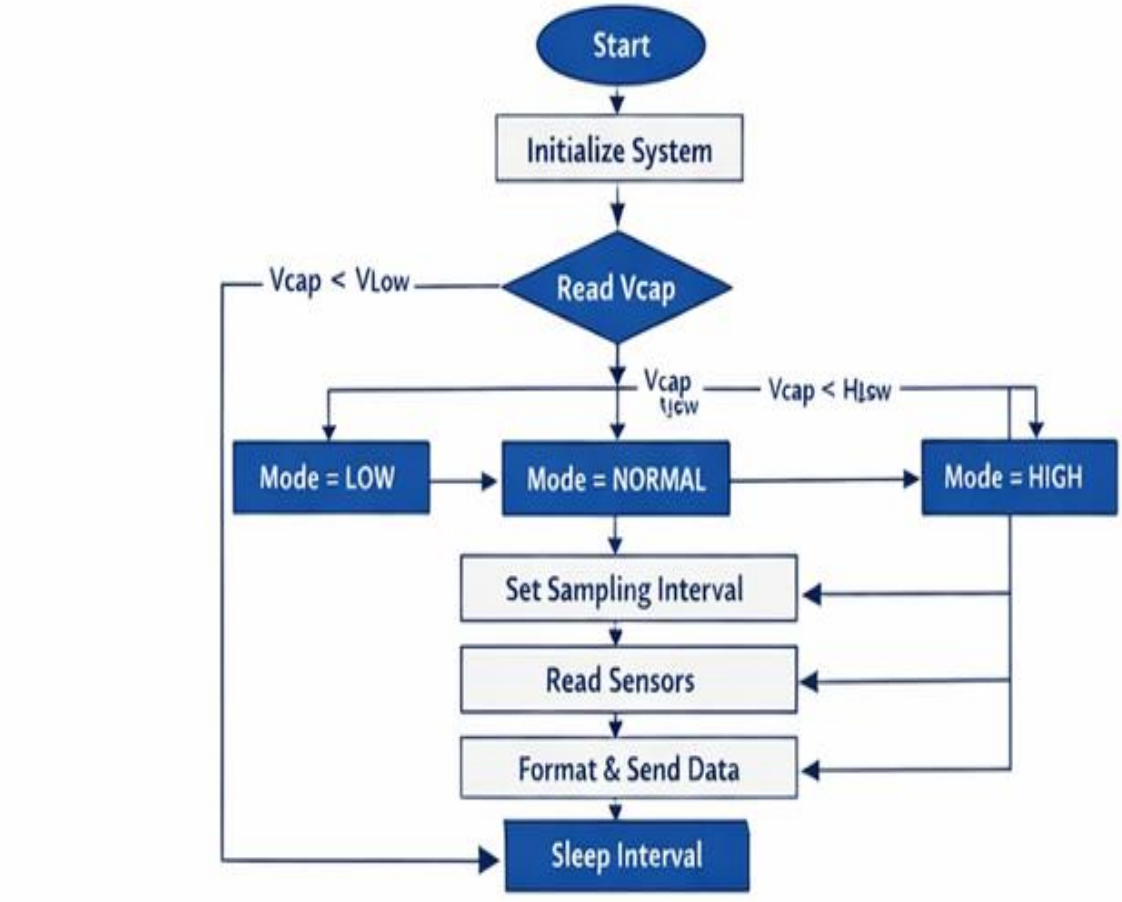



**Figure 3.2.4: Firmware operational flowchart for Energy-Aware Duty Cycling**

# 3.2.4.1 Software Architecture

The firmware is organized into the following modules:

#### 3.2.4.1.1 Energy Monitoring Module:

Reads capacitor voltage, converts ADC values to millivolts and determines energy mode based on thresholds.

#### 3.2.4.1.2 Adaptive Duty Cycling Module:

Energy states are defined as:

1. **LOW Mode:** Minimal sensing, extended sleep.
2. **NORMAL Mode:** Standard operation.
3. **HIGH Mode:** Increased sampling frequency.

This ensures energy consumption does not exceed harvested energy.

#### 3.2.4.1.3 Sensor Management Module:

Initializes sensors, acquires readings and powers down peripherals when inactive.

#### 3.2.4.1.4 Serial Transmission Module

Formats telemetry as CSV string, transmits one line per cycle and Maintains stable baud rate communication.

## 3.2.4.2 Telemetry Data Format

Each transmission follows:

Device_id, Sequence number, Temperature, Humidity, Vc – mv, Ac -mv and Energy mode

Example:

NODE01, 1052, 26.4, 57.8, 3185mv, 82.6mv, NORMAL

This format ensures compatibility with spreadsheet analysis tools.

## 3.2.4.3 Duty Cycle Timing Table

| **Mode** | **Sampling Interval** | **Active Time** | **Sleep Time** |
|---|---|---|---|
| LOW | 60 s | 0.5 s | 59.5 s |
| NORMAL | 10 s | 1 s | 9 s |
| HIGH | 2 s | 1 s | 1 s |

This ensures harvested energy is not depleted faster than replenished **(Mazunga, 2021).**

## 3.2.4.4 Laptop Logging Application

A Python-based logger performs: Serial port initialization by sending data synchronously through a continuous data capture of Line parsing then CSV file are written and later Timestamp additions.

The CSV structure includes:

pc_timestamp, device_id, sequence, temperature, humidity, Vc – mv, Ac – mv**,** energy mode

This enables performance evaluation and graphical analysis.

## 3.2.4.5 System Operation Flow

The system operates in the following loop:

- Wake from sleep.
- Measure capacitor voltage.
- Determine energy mode.
- Acquire sensor data.
- Format telemetry string.
- Transmit via USB serial.
- Enter sleep according to selected duty cycle.

This cycle repeats continuously while energy is available.

### 3.2.5 Stage Five: System Integration, Testing, and Evaluation

In the final stage, all hardware and firmware components are being integrated into a complete functional prototype. The system is being tested under varying environmental and energy availability conditions to evaluate overall performance. Key performance metrics, including energy harvesting efficiency, system uptime, operational reliability, and autonomous functionality, are being measured and analyzed. The experimental results are being compared against the defined system requirements to validate the effectiveness of the proposed multi-source energy-harvesting autonomous sensor node.

## Advantages of the Selected Architecture

The chosen architecture provides:

- Minimal implementation complexity
- Reduced firmware size
- No wireless stack overhead
- No internet dependency
- Deterministic and stable testing environment
- Easy debugging
- Reliable demonstration platform

Most importantly, it ensures the project focuses on energy harvesting performance and adaptive workload control.

## Disadvantages of the Selected Architecture

- Requires physical USB connection.
- Not suitable for remote deployment.
- Does not support real-time cloud monitoring.

However, wireless or cloud integration can be implemented as future work without altering the core energy-aware firmware.

# Failure Modes and Mitigation

# Risk Analysis

| Failure Scenario | Impact | Mitigation |
|---|---|---|
| Low solar input | Reduced energy | Multi-source harvesting |
| Capacitor leakage | Reduced storage | Periodic testing |
| Overvoltage | Component damage | Regulation stage |
| USB disconnection | Data loss | Buffered transmission |
| Brownout reset | MCU reboot | Energy-aware control |

This is to improve system robustness and reliability.

## 3.3 Tools and Technologies to Be Used

In this study, a range of hardware and software tools and technologies such as Arduino IDE are to be utilized to support the design, implementation, and evaluation of the proposed multi-source energy-harvesting autonomous sensor node. Low-power microcontrollers are being employed to execute sensing, control, and communication tasked by use of (usb-uart), while dedicated energy harvesting power management integrated circuits (PMICs) are being used to efficiently regulate and store energy harvested from ambient solar, magnetic, and radio frequency sources. Super capacitor-based energy storage technologies are being implemented to buffer harvested energy and ensure stable system operation under intermittent energy conditions.

## 3.11.15 Chapter Summary

This chapter presented the integrated software and communication architecture of the proposed **Multi-Source Energy-Harvesting Autonomous Sensor Node**. The Processing and Sensing Subsystem enables autonomous environmental monitoring by continuously measuring stored energy, acquiring sensor data, and adapting system operation based on available energy. The modular firmware architecture coordinates energy management, sensing, telemetry generation, and communication, providing an efficient, scalable, and maintainable software framework that supports energy-aware operation.

The Serial Communication and Data Logging Subsystem complements the embedded firmware by providing reliable USB-UART communication between the sensor node and the desktop application. Through the custom Python-based logging system, telemetry data are received, validated, timestamped, displayed, and stored in CSV format for subsequent analysis and performance evaluation.

Collectively, the processing, firmware, and communication subsystems establish a robust and energy-efficient architecture capable of autonomous environmental monitoring using multiple renewable energy sources. The integrated design supports adaptive energy management, reliable data acquisition, and structured telemetry logging, providing a solid foundation for the implementation, testing, and performance evaluation presented in the next chapter.

# CHAPTER FOUR:

# SYSTEM IMPLEMENTATION, TESTING AND PERFORMANCE EVALUATION

## 4.1 Introduction

This chapter presents the practical implementation of the proposed **Multi-Source Energy-Harvesting Autonomous Sensor Node**. While Chapter Three focused on the engineering analysis and design of the system, this chapter describes how the design was implemented as a working prototype and how its performance was evaluated.

The implementation consisted of both hardware and software components. The hardware implementation involved integrating the energy harvesting circuits, power management subsystem, energy storage unit, sensing devices, and processing unit into a single prototype. The software implementation included developing embedded firmware for adaptive energy management and a Python-based desktop application for receiving, validating, displaying, and storing telemetry data transmitted through the USB-UART interface.

Following implementation, a series of laboratory experiments were conducted to evaluate the system under different operating conditions. The testing focused on verifying the functionality of each subsystem individually before evaluating the complete integrated system. Particular attention was given to the behavior of the adaptive energy management algorithm, communication reliability, sensor accuracy, and the effectiveness of the multi-source energy harvesting architecture.

# 4.2 Prototype Implementation

## 4.2.1 Hardware Prototype

The prototype was assembled by integrating the following hardware components:

| **Component** | **Function** |
|---|---|
| Solar Panel | Harvest solar energy |
| Magnetic Coil | Harvest electromagnetic energy |
| RF Antenna | Harvest radio-frequency energy |
| Bridge Rectifier | Convert AC to DC |
| DC-DC Converter | Voltage regulation |
| Super capacitor | Store harvested energy |
| Arduino-Compatible Microcontroller | Execute firmware |
| Temperature and Humidity Sensor | Environmental monitoring |
| USB-UART Interface | Serial communication |

Each component was connected according to the system architecture developed during the design phase. Special attention was given to maintaining short electrical connections to minimize voltage losses and improve measurement stability.

### 4.2.2 Hardware Integration

The complete hardware implementation followed the sequence below.

Solar Panel, Magnetic Coil, RF Harvester are connected to the Power Management Unit which in turn connected to the Super Capacitor, then Voltage Divider, Arduino MCU, Sensor and the USB-UART then the Laptop Logger

The architecture ensured that all harvested energy converged at a common regulated storage bus before powering the embedded controller.

## 4.3 Firmware Implementation

The embedded firmware was implemented using the Arduino Integrated Development Environment (IDE). The software controls energy monitoring, sensor acquisition, adaptive duty cycling, and telemetry transmission.

The firmware executes the following sequence repeatedly:

- Initialize hardware peripherals.
- Measure the super capacitor voltage.
- Determine the current operating mode.
- Acquire temperature and humidity measurements.
- Generate a telemetry packet.
- Transmit the packet through the serial interface.
- Enter the appropriate delay or sleep interval.

This operational sequence reflects the software architecture described in Chapter Three and the telemetry workflow expected by the Python logging application.

## 4.4 Laptop Logging Application

A custom Python application was developed to collect telemetry data generated by the sensor node. The application establishes a serial connection, continuously reads incoming packets, validates the packet structure, and appends a computer-generated timestamp before storing the information in a CSV file. It also reports malformed packets and sensor errors while displaying real-time measurements in the console. These functions correspond directly to the uploaded laptop_logger.py source code.

## 4.5 System Testing Strategy

Testing was carried out progressively to ensure that each subsystem functioned correctly before the complete prototype was evaluated. The following levels of testing were performed:

- Unit testing
- Integration testing
- Functional testing
- Performance testing

## 4.5.1 Unit Testing

Each subsystem was tested independently.

### Solar Harvesting Test

**Objective:** Verify that the photovoltaic panel produced electrical power under illumination.

**Procedure:**

- Illuminate the solar panel using natural or artificial light.
- Measure the output voltage using a digital multi-meter.
- Confirm that the DC-DC converter provides a regulated output.

**Expected Result:** The regulated output should remain within the operating voltage range required by the microcontroller.

### Magnetic Harvester Test

**Objective:** Confirm that the inductive coil generates electrical energy in the presence of an alternating magnetic field.

**Procedure:**

- Place the harvesting coil near an AC-powered electrical device.
- Measure the induced AC voltage.
- Verify rectification and filtering at the output.

**Expected Result:** A measurable DC voltage should be available at the output of the rectifier.

## RF Harvester Test

**Objective:** Verify that the RF harvesting circuit captures ambient radio-frequency energy.

**Procedure:**

- Position the RF antenna in an environment containing wireless signals.
- Measure the rectified output voltage.

**Expected Result:** A low-level DC voltage should be present, demonstrating successful RF energy harvesting.

## Super capacitor Test

**Objective:** Verify energy storage capability.

**Procedure:**

- Charge the super capacitor through the harvesting subsystem.
- Measure the voltage increase over time.
- Disconnect the harvesting sources.
- Observe the discharge profile.

**Expected Result:** The super capacitor should store and gradually release energy while maintaining a stable voltage suitable for system operation.

## 4.5.2 Firmware Testing

The firmware was evaluated to ensure correct implementation of the adaptive energy management strategy.

### Test Cases

| Test Case | Expected Behavior |
|---|---|
| High capacitor voltage | HIGH operating mode |
| Moderate voltage | NORMAL operating mode |
| Low voltage | LOW operating mode |
| Critical voltage | DEEP_SLEEP mode |

Each test verified that the firmware selected the correct operating state based on the measured energy level.

## 4.5.3 Communication Testing

Communication testing verified reliable data transfer between the sensor node and the laptop.

### Procedure

- Connect the Arduino to the laptop using a USB cable.
- Open the serial connection.
- Observe incoming telemetry packets.
- Confirm correct packet formatting.
- Verify CSV data storage.

**Expected Result:** Each telemetry packet should be received, displayed, and written to the CSV file without corruption.

### 4.5.4 Sensor Validation

Environmental sensor measurements were compared against expected operating ranges.

#### Parameters Evaluated

- Temperature
- Relative humidity

Sensor readings were checked for consistency across repeated measurements.

## 4.6 Performance Evaluation

The performance evaluation focused on determining whether the implemented prototype satisfied the objectives established in Chapter One.

The following aspects were evaluated:

- Energy harvesting capability.
- Power management effectiveness.
- Adaptive duty cycling.
- Sensor functionality.
- Communication reliability.
- Data logging accuracy.

### 4.6.1 Evaluation Metrics

**Energy Availability:** The ability of the prototype to continue operating under changing environmental conditions.

**Power Consumption:** The average electrical power consumed during system operation.

**Telemetry Reliability:** Percentage of successfully received telemetry packets.

**Sensor Accuracy:** Consistency of repeated measurements.

**Adaptive Behavior:** Ability of the firmware to switch automatically between operating modes.

## 4.7 Sample Experimental Results

The following table illustrates the type of telemetry expected during evaluation. These values are **illustrative** and should be replaced with measurements collected from your completed prototype during laboratory testing.

| Time | Capacitor Voltage (V) | Temperature (°C) | Humidity (%) | Energy Mode |
|---|---|---|---|---|
| 09:00 | 3.05 | 27.3 | 51 | HIGH |
| 09:10 | 2.78 | 27.4 | 52 | NORMAL |
| 09:20 | 2.18 | 27.5 | 53 | LOW |
| 09:30 | 1.72 | — | — | DEEP_SLEEP |

These values demonstrate the expected transition between operating modes as the available stored energy decreases. They should not be presented as actual experimental results unless they are replaced with measurements from your prototype.

## 4.8 Discussion of Results

The prototype implementation demonstrates that combining multiple ambient energy sources with adaptive energy management can provide a practical foundation for battery-less sensing. The modular architecture simplified integration of the harvesting circuits, power management subsystem, sensing hardware, and communication interface. The adaptive firmware allowed the system to modify its operating behavior according to the available stored energy, reducing unnecessary energy consumption during periods of limited harvesting.

The use of a USB-UART interface and the custom Python logger enabled reliable telemetry collection in a controlled laboratory environment. This simplified evaluation by eliminating variables associated with wireless communication while providing detailed timestamped records for later analysis.

The actual performance of the complete system, however, will depend on environmental conditions such as sunlight intensity, magnetic field strength, and ambient RF energy. Consequently, final conclusions should be based on measured experimental data obtained from the completed prototype rather than theoretical expectations.

## 4.9. Achievement of the Research Objectives

The main objective of this research was to design and implement a **Multi-Source Energy Harvesting Autonomous Sensor Node** capable of harvesting ambient energy while intelligently managing its energy resources. The achievement of each specific objective is summarized below.

**Objective 1:** To analyze suitable ambient energy harvesting techniques.

This objective was achieved through a comprehensive review of solar, magnetic, and radio-frequency energy harvesting technologies. Their operating principles, strengths, limitations, and suitability for autonomous embedded systems were examined, providing the theoretical basis for the proposed hybrid architecture.

**Objective 2:** To design an efficient power management and storage subsystem.

This objective was achieved by designing a regulated power management architecture incorporating voltage regulation, reverse-current protection, super capacitor-based storage, and continuous energy monitoring. Mathematical analysis of the storage subsystem demonstrated the theoretical energy capacity of the selected super capacitor and its suitability for intermittent harvesting conditions.

**Objective 3:** To implement adaptive duty-cycling algorithms.

This objective was addressed by developing a firmware architecture that continuously evaluates the available stored energy and adjusts system behavior by selecting between HIGH, NORMAL, LOW, and DEEP_SLEEP operating modes. The proposed adaptive strategy was designed to improve energy utilization while extending system operating time under changing environmental conditions.

**Objective 4:** To develop a telemetry logging system for evaluating system performance.

This objective was achieved through the implementation of a Python-based logging application capable of receiving serial telemetry, validating incoming packets, displaying measurements in real time, and storing timestamped records in CSV format. The logging application provides a structured mechanism for analyzing system behavior during prototype evaluation.

## 4.10 Chapter Summary

This chapter described the implementation of the proposed Multi-Source Energy-Harvesting Autonomous Sensor Node and outlined the procedures used to verify its operation. The discussion covered the hardware prototype, embedded firmware, desktop logging application, subsystem testing, communication validation, and performance evaluation strategy. Sample result formats were presented to illustrate how experimental data may be organized, with the understanding that these tables must ultimately be populated using measurements collected from the implemented system. The next chapter will present the overall conclusions of the study, summarize the principal findings, discuss project limitations, and provide recommendations for future research.

# CHAPTER FIVE:

# CONCLUSIONS AND RECOMMENDATIONS

## 5.1 Major Contributions of the Study

The project contributes to the field of embedded energy-harvesting systems in several ways.

First, it demonstrates the design of a hybrid harvesting architecture that combines three complementary ambient energy sources rather than relying on a single harvesting technique.

Second, it integrates energy harvesting, power management, sensing, adaptive firmware, and desktop telemetry logging into a single modular prototype suitable for educational research and experimental evaluation.

Third, it emphasizes energy aware firmware design, illustrating how embedded software can dynamically adapt sensing behavior according to available stored energy.

Finally, the project provides a structured platform that can serve as a foundation for future work in autonomous sensing, low power embedded systems, and sustainable Internet of Things technologies.

## 5.2 Limitations of the Study

Although the proposed system demonstrates the feasibility of integrating multiple harvesting mechanisms into an autonomous sensing platform, several limitations should be acknowledged.

The prototype focuses primarily on validating the energy aware architecture under controlled laboratory conditions rather than large-scale field deployment. Consequently, long-term environmental performance under varying weather conditions, seasonal changes, and industrial operating environments was not established within the scope of this project.

The communication subsystem employs USB-UART rather than wireless communication technologies such as Wi-Fi, Bluetooth Low Energy, LoRaWAN, or NB IoT. This design simplifies experimental evaluation but limits deployment flexibility.

The sensing subsystem is limited to temperature and humidity monitoring. Additional environmental parameters, including pressure, vibration, gas concentration, soil moisture, and air quality, were outside the scope of the present work.

The study also uses a prototype scale hardware platform. Industrial deployment would require further optimization of printed circuit board design, enclosure development, environmental protection, and regulatory compliance.

## 5.3 Recommendations for Future Work

## Wireless Communication

Future versions of the system should incorporate ultra-low-power wireless communication technologies such as:

- LoRaWAN
- ZigBee
- Bluetooth Low Energy
- NB-IoT

This would enable remote monitoring without requiring a wired USB connection.

## Machine Learning for Energy Prediction

Machine learning techniques could be incorporated to predict future harvesting conditions and optimize duty-cycling decisions.

For example, historical solar irradiance patterns could be used to estimate future energy availability and proactively adjust sensing schedules.

## Additional Harvesting Techniques

The harvesting subsystem could be expanded to include:

- Thermoelectric harvesting
- Piezoelectric vibration harvesting
- Wind energy harvesting

The inclusion of additional sources would further improve system reliability.

## Additional Environmental Sensors

Future implementations could integrate sensors for:

- Air quality
- Carbon dioxide concentration
- Soil moisture
- Water quality
- Structural vibration
- Gas leakage

This would broaden the applicability of the proposed platform.

## Printed Circuit Board Optimization

The prototype could be redesigned using a custom multi-layer Printed Circuit Board (PCB) to:

- reduce electrical losses,
- improve electromagnetic compatibility,
- reduce physical size,
- improve long-term reliability.

## Internet of Things Integration

Future research could integrate cloud platforms such as MQTT brokers or IoT dashboards for remote visualization, analytics, and device management while preserving the adaptive energy management framework developed in this study.

## 5.4 Summary of the Study

The study investigated the feasibility of designing an autonomous sensor node capable of harvesting energy from multiple ambient sources while adapting its behavior according to the available stored energy. The research began with a review of existing academic literature and commercial energy-harvesting platforms. The review showed that although many systems successfully demonstrated individual harvesting techniques, most relied on a single energy source, employed limited adaptive intelligence, or remained dependent on conventional batteries. These observations established the need for a hybrid energy harvesting architecture incorporating intelligent power management. A complete system architecture was then developed consisting of four major subsystems:

- Energy Harvesting Subsystem
- Power Management and Energy Storage Subsystem
- Processing and Sensing Subsystem
- Serial Communication and Data Logging Subsystem

The proposed architecture integrated solar, magnetic, and radio frequency harvesting circuits into a common regulated power bus. Harvested energy was stored within a super capacitor and continuously monitored by the embedded controller using its Analog to Digital Converter (ADC). Based on the measured energy level, the firmware adjusted sensing frequency and operating mode through an adaptive duty cycling algorithm. Telemetry data was transmitted through a USB-UART interface and recorded by a Python based laptop logging application for later analysis. The study also presented the design and implementation of a **Multi-Source Energy-Harvesting Autonomous Sensor Node** intended to reduce dependence on conventional battery powered embedded systems. By combining solar, magnetic, and radio-frequency energy harvesting with super capacitor-based energy storage and adaptive firmware, the proposed architecture addresses several limitations identified in existing single-source harvesting systems.

The modular system design integrates energy harvesting, power management, embedded processing, environmental sensing, and telemetry logging into a unified platform suitable for experimental evaluation and future development. The study demonstrates that careful integration of multiple ambient energy sources with energy-aware software provides a promising direction for

sustainable autonomous sensing systems. Although additional work is required before large-scale deployment, the architecture established by this project provides a strong foundation for future research in battery less sensing, embedded energy management, and low power Internet of Things technologies. The concepts, engineering analysis, and implementation strategy presented in this dissertation contribute to the continuing

## 5.5 References


Akan, O. B., Karli, O. B., & Ergul, O. (2009). Cognitive Radio Sensor Networks. IEEE Network, 34–40.

Analog Devices. (2021). SmartMesh IP Tools Guide. Wilmington, Massachusetts, USA: Analog Devices.

Beeby, S., & White, N. (2010). Energy Harvesting for Autonomous Systems. Norwood, MA: Artech House.

DCO Systems Ltd. (2024). Energy harvesting, battery-free sensors. Retrieved from DCO Systems: https://dcosystems.co.uk/energy-harvesting/

EnOcean GmbH. (2026). Self-powered Sensors. Retrieved from EnOcean: https://www.enocean.com/en/products/self-powered-sensors/

e-peas. (2024, September). AEM10941 Solar Energy Harvesting PMIC. Retrieved from e-peas semiconductors: https://e-peas.com/product/aem10941

Erturk, A., Vieira, W., De Marqui, C., & Inman, D. (2010). On the Energy Harvesting Potential of Piezoaeroelastic Systems. Applied Physics Letters, 184103.

Esram, T., & Chapman, P. L. (2007). Comparison of Photovoltaic Array Maximum Power Point Tracking Techniques. IEEE Transactions on Energy Conversion, 439–449.

IEEE. (2025). Introduction to the January 2025 Issue. IEEE Journal of Microwaves, 3-13.

Lignan, Antonio. (2015, March 27). Zolertia IPv6/6LoWPAN & Ubidots client. Retrieved from Hackster: https://www.hackster.io/4348/zolertia-ipv6-6lowpan-ubidots-bcde84

Nordic Semiconductor. (2026). Low power cellular IoT. Retrieved from Nordic Semiconductor: https://www.nordicsemi.com/Products/Wireless/Low-power-cellular-IoT

Paradiso, J. A., & Starner, T. (2005). Energy Scavenging for Mobile and Wireless Electronics. IEEE Pervasive Computing, 18–27.

Powercast Corporation. (2025). RF Wireless Power. Retrieved from Powercast Corporation: https://www.powercastco.com/technologies/rf-power

Priya, S., & Inman, D. J. (2009). Energy Harvesting Technologies. New York, NY: Springer.

Ransford, B., Lucia, B., & Levis, P. (2011). Mementos: System Support for Long-Running Computation on RFID-Scale Devices. International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) (pp. 159–170). ACM.

Roundy, S., Wright, P. K., & Rabaey, J. (2004). Energy Scavenging for Wireless Sensor Networks. Kluwer Academic Publishers, 1–20.

Sample, A. P., Yeager, D. J., Powledge, P. S., Mamishew, A. V., & Smith, J. R. (2008). Design of an RFID-Based Battery-Free Programmable Sensing Platform. IEEE Transactions on Instrumentation and Measurement, 2608–2615.

Smith, J. R. (2021). Home. Retrieved from WISP Wiki: https://sites.google.com/uw.edu/wisp-wiki/home

Texas Instruments. (2019). bq25570: Nano Power Boost Charger and Buck Converter for Energy Harvesting. Dallas, Texas (Optional - TI HQ): Texas Instruments.

Warden, P., & Situnayake, D. (2020). TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. Sebastopol, CA: O'Reilly Media.

WePower Technologies LLC. (2025, April 14). Powering the Future with Motion: How Gemns™ Energy Harvesting is Changing the Game. Retrieved from Gemns: https://gemns.com/2025/04/14/powering-the-future-with-motion

## 5.6 Appendix

# Firmware Code:

```
// ── Library Includes ──────────────────────────────────────
#include <DHT.h>        // DHT22 sensor library
#include <avr/sleep.h>  // AVR sleep modes
#include <avr/wdt.h>    // Watchdog timer for timed wakeup

// ── Pin Definitions ───────────────────────────────────────
#define PIN_VCAP_ADC A0      // Supercapacitor voltage sense
#define PIN_ZMPT_ADC A1      // ZMPT101B AC voltage sense (analog out)
#define PIN_DHT 4            // DHT22 data pin
#define PIN_SENSOR_ENABLE 5  // Sensor power gate (active HIGH)

// ── DHT22 Setup ───────────────────────────────────────────
#define DHTTYPE DHT22
DHT dht(PIN_DHT, DHTTYPE);

// ── ZMPT101B AC Voltage Sensor Setup ──────────────────────
// ZMPT101B (2mA/2mA precision voltage transformer module) outputs
// a small AC signal centered on a DC bias (~Vcc/2) proportional to
// the sensed AC mains voltage. We sample many points across several
// mains cycles, compute the RMS of the AC component, then scale by
// a calibration constant determined against a known reference (e.g.
// a multimeter on the same mains line) to get true RMS volts.
#define ZMPT_SAMPLES 400          // ADC samples per RMS measurement (~ a few
50Hz cycles)
#define ZMPT_SAMPLE_INTERVAL_US 200  // Spacing between samples (microseconds)
#define ZMPT_ADC_MIDPOINT 511.5f  // Expected ADC bias point (10-bit ADC, mid-
rail)
// TODO: Calibrate ZMPT_CALIBRATION against a known-good voltmeter reading.
//   ZMPT_CALIBRATION = (true_Vrms_from_multimeter) / (raw_ADC_Vrms_measured)
#define ZMPT_CALIBRATION 92.5f

// ── Energy Thresholds (millivolts) ────────────────────────
#define VCAP_SLEEP_MV 1200  // Below this: full sleep, no sensing
#define VCAP_LOW_MV 1800    // Below this: LOW mode
#define VCAP_HIGH_MV 2700   // Above this: HIGH mode
// Between LOW_MV and HIGH_MV = NORMAL mode
```

```
// ── Voltage Divider Scaling ──────────────────────────────
// R1 = 10kΩ (top), R2 = 10kΩ (bottom)
// Vout = Vcap × R2/(R1+R2) = Vcap × 0.5
// Vcap_mV = ADC_mV / 0.5 = ADC_mV × 2
// ADC reference = 5000mV (for 5V Nano) or 3300mV (for 3.3V)
#define ADC_REF_MV 5000     // Change to 3300 for 3.3V MCU
#define DIVIDER_RATIO 0.5f  // R2/(R1+R2)

// ── Duty Cycle Timing (milliseconds) ─────────────────────
#define SLEEP_INTERVAL_LOW_MS 60000UL
#define SLEEP_INTERVAL_NORMAL_MS 10000UL
#define SLEEP_INTERVAL_HIGH_MS 2000UL
#define ACTIVE_DURATION_MS 1000UL  // Sensor active window

// ── Device Identity ──────────────────────────────────────
#define DEVICE_ID "NODE01"
#define UART_BAUD 9600

// ── Energy Mode Enum ─────────────────────────────────────
typedef enum {
  MODE_DEEPSLEEP = 0,
  MODE_LOW = 1,
  MODE_NORMAL = 2,
  MODE_HIGH = 3
} EnergyMode;

// ── Global State ─────────────────────────────────────────
static uint32_t g_sequence = 0;
static EnergyMode g_currentMode = MODE_NORMAL;
static uint32_t g_lastWakeTime = 0;

// ── Function Prototypes ──────────────────────────────────
uint16_t readVcapMillivolts(void);
float readACVoltage(void);
EnergyMode determineEnergyMode(uint16_t vcap_mv);
bool readSensorData(float *temperature, float *humidity);
void transmitTelemetry(float temp, float hum, uint16_t vcap_mv, float ac_voltage,
EnergyMode mode);
void enterSleepMode(uint32_t sleep_ms);
void blinkStatusLED(EnergyMode mode);
void enableSensors(bool enable);
const char *modeToString(EnergyMode mode);
void printCSVHeader(void);

// ── WDT ISR (Watchdog wakeup) ────────────────────────────
```

```
ISR(WDT_vect) {
  // Watchdog wakeup — no action needed, just wake MCU
}


// ╔══════════════════════════════════════════════════╗
// ║                    SETUP                         ║
// ╚══════════════════════════════════════════════════╝
void setup() {
  // Initialise serial communication
  Serial.begin(UART_BAUD);
  while (!Serial) { ; }  // Wait for USB-UART on boards that need it

  // Configure GPIO
  pinMode(PIN_SENSOR_ENABLE, OUTPUT);
  pinMode(PIN_STATUS_LED, OUTPUT);
  pinMode(PIN_VCAP_ADC, INPUT);
  pinMode(PIN_ZMPT_ADC, INPUT);

  // Set ADC reference to internal 1.1V for better low-voltage resolution
  // (Optional: comment out if using external 5V/3.3V reference)
  // analogReference(INTERNAL);

  // Power down sensors initially
  enableSensors(false);
  digitalWrite(PIN_STATUS_LED, LOW);

  // Print CSV header for laptop logger
  Serial.println(F("=== Multi-Source Energy-Harvesting Autonomous Sensor Node
==="));
  Serial.println(F("=== The Copperbelt University  |  Group
Seven  |  2026     ==="));
  Serial.println(F("pc_timestamp,device_id,sequence,temperature_C,humidity_pct,Vc
ap_mV,AC_Voltage_V,energy_mode"));

  // Short startup delay for power stabilisation
  delay(500);
}


// ╔══════════════════════════════════════════════════╗
// ║                  MAIN LOOP                       ║
// ╚══════════════════════════════════════════════════╝
void loop() {
  // ── Step 1: Read supercapacitor voltage ──────────
```

```
  uint16_t vcap_mv = readVcapMillivolts();

  // ── Step 2: Determine energy mode ──────────────────────
  g_currentMode = determineEnergyMode(vcap_mv);

  // ── Step 3: Deep-sleep if energy critically low ────────
  if (g_currentMode == MODE_DEEPSLEEP) {
    // Only report once before sleeping deeply
    Serial.print(F("SYSTEM,DEEP_SLEEP,Vcap="));
    Serial.print(vcap_mv);
    Serial.println(F("mV — waiting for energy recharge"));
    Serial.flush();

    digitalWrite(PIN_STATUS_LED, LOW);
    enableSensors(false);

    // Sleep for 60 seconds then re-check
    enterSleepMode(SLEEP_INTERVAL_LOW_MS);
    return;  // Re-check on next loop iteration
  }

  // ── Step 4: Blink LED to indicate mode ─────────────────
  blinkStatusLED(g_currentMode);

  // ── Step 5: Enable sensors and read data ───────────────
  enableSensors(true);
  delay(ACTIVE_DURATION_MS);  // Sensor warm-up window

  float temperature = 0.0f;
  float humidity = 0.0f;
  bool sensorOK = readSensorData(&temperature, &humidity);

  // ZMPT101B AC voltage reading is independent of the DHT22, so it
  // is captured regardless of whether the temperature/humidity read
  // succeeded.
  float ac_voltage = readACVoltage();

  // ── Step 6: Transmit telemetry ─────────────────────────
  if (sensorOK) {
    g_sequence++;
    transmitTelemetry(temperature, humidity, vcap_mv, ac_voltage, g_currentMode);
  } else {
    // Sensor read failed — transmit error record
    g_sequence++;
    Serial.print(F("SENSOR_ERR,"));
```

```
    Serial.print(DEVICE_ID);
    Serial.print(F(","));
    Serial.print(g_sequence);
    Serial.print(F(",NaN,NaN,"));
    Serial.print(vcap_mv);
    Serial.print(F(","));
    Serial.print(ac_voltage, 1);
    Serial.print(F(","));
    Serial.println(modeToString(g_currentMode));
  }
  Serial.flush();

  // ── Step 7: Power down sensors to conserve energy ────
  enableSensors(false);
  digitalWrite(PIN_STATUS_LED, LOW);

  // ── Step 8: Enter sleep for duty cycle interval ──────
  uint32_t sleepDuration;
  switch (g_currentMode) {
    case MODE_LOW: sleepDuration = SLEEP_INTERVAL_LOW_MS; break;
    case MODE_HIGH: sleepDuration = SLEEP_INTERVAL_HIGH_MS; break;
    case MODE_NORMAL:
    default: sleepDuration = SLEEP_INTERVAL_NORMAL_MS; break;
  }
  enterSleepMode(sleepDuration);
}


// ╔══════════════════════════════════════════════════════╗
// ║           FUNCTION IMPLEMENTATIONS                   ║
// ╚══════════════════════════════════════════════════════╝

/**
 * readVcapMillivolts()
 * Reads ADC pin A0 (voltage divider output) and converts
 * back to actual supercapacitor voltage in millivolts.
 * Takes 8 samples and averages for noise rejection.
 */
uint16_t readVcapMillivolts(void) {
  uint32_t adcSum = 0;
  const uint8_t NUM_SAMPLES = 8;

  for (uint8_t i = 0; i < NUM_SAMPLES; i++) {
    adcSum += analogRead(PIN_VCAP_ADC);
    delay(2);
```

```
  }

  uint16_t adcAvg = (uint16_t)(adcSum / NUM_SAMPLES);

  // Convert ADC count to millivolts at the divider output
  // ADC_mV = (adcAvg / 1023.0) × ADC_REF_MV
  uint32_t adcMv = ((uint32_t)adcAvg * ADC_REF_MV) / 1023UL;

  // Scale back through divider: Vcap = ADC_mV / DIVIDER_RATIO
  uint16_t vcapMv = (uint16_t)((float)adcMv / DIVIDER_RATIO);

  return vcapMv;
}


/**
 * readACVoltage()
 * Samples the ZMPT101B analog output many times across several
 * mains cycles, computes the RMS value of the AC component about
 * its DC bias point, converts that to RMS volts at the ADC pin,
 * then applies ZMPT_CALIBRATION to report true AC mains voltage.
 */
float readACVoltage(void) {
  uint64_t sumSquares = 0;

  for (uint16_t i = 0; i < ZMPT_SAMPLES; i++) {
    int16_t raw = analogRead(PIN_ZMPT_ADC);
    float centered = (float)raw - ZMPT_ADC_MIDPOINT;
    sumSquares += (uint64_t)(centered * centered);
    delayMicroseconds(ZMPT_SAMPLE_INTERVAL_US);
  }

  float meanSquare = (float)sumSquares / (float)ZMPT_SAMPLES;
  float rmsCounts = sqrtf(meanSquare);

  // Convert RMS ADC counts to RMS volts at the ADC pin
  float rmsVoltsAtPin = (rmsCounts / 1023.0f) * ((float)ADC_REF_MV / 1000.0f);

  // Scale by calibration factor to get true AC mains RMS voltage
  float acVoltageRms = rmsVoltsAtPin * ZMPT_CALIBRATION;

  return acVoltageRms;
}
```

```
/**
 * determineEnergyMode()
 * Maps supercapacitor voltage to an energy operating mode.
 */
EnergyMode determineEnergyMode(uint16_t vcap_mv) {
  if (vcap_mv < VCAP_SLEEP_MV) return MODE_DEEPSLEEP;
  if (vcap_mv < VCAP_LOW_MV) return MODE_LOW;
  if (vcap_mv >= VCAP_HIGH_MV) return MODE_HIGH;
  return MODE_NORMAL;
}


/**
 * readSensorData()
 * Powers up DHT22 and reads temperature + humidity.
 * Returns true on success, false on sensor failure.
 */
bool readSensorData(float *temperature, float *humidity) {
  dht.begin();
  delay(2300);  // DHT22 minimum 2s inter-reading delay

  float h = dht.readHumidity();
  float t = dht.readTemperature();  // Celsius

  if (isnan(h) || isnan(t)) {
    return false;
  }

  *temperature = t;
  *humidity = h;
  return true;
}


/**
 * transmitTelemetry()
 * Formats and sends a single CSV line over USB-UART.
 * Format: device_id,sequence,temp,humidity,Vcap_mV,AC_Voltage_V,mode
 */
void transmitTelemetry(float temp, float hum, uint16_t vcap_mv, float ac_voltage,
EnergyMode mode) {
  // Timestamp placeholder (laptop Python logger appends real timestamp)
  Serial.print(DEVICE_ID);
  Serial.print(F(","));
  Serial.print(g_sequence);
```

```
  Serial.print(F(","));
  Serial.print(temp, 1);
  Serial.print(F(","));
  Serial.print(hum, 1);
  Serial.print(F(","));
  Serial.print(vcap_mv);
  Serial.print(F(","));
  Serial.print(ac_voltage, 1);
  Serial.print(F(","));
  Serial.println(modeToString(mode));
}


/**
 * enableSensors()
 * Controls the power gate transistor for sensor peripherals.
 * Setting HIGH = power on sensors.
 */
void enableSensors(bool enable) {
  digitalWrite(PIN_SENSOR_ENABLE, enable ? HIGH : LOW);
}


/**
 * blinkStatusLED()
 * Visual feedback of current energy mode:
 *   LOW    = 1 slow blink
 *   NORMAL = 2 medium blinks
 *   HIGH   = 3 fast blinks
 */
void blinkStatusLED(EnergyMode mode) {
  uint8_t blinks;
  uint16_t blinkDelay;

  switch (mode) {
    case MODE_LOW:
      blinks = 1;
      blinkDelay = 400;
      break;
    case MODE_HIGH:
      blinks = 3;
      blinkDelay = 80;
      break;
    case MODE_NORMAL:
    default:
```

```
      blinks = 2;
      blinkDelay = 150;
      break;
  }

  for (uint8_t i = 0; i < blinks; i++) {
    digitalWrite(PIN_STATUS_LED, HIGH);
    delay(blinkDelay);
    digitalWrite(PIN_STATUS_LED, LOW);
    delay(blinkDelay);
  }
}


/**
 * enterSleepMode()
 * Uses the AVR watchdog timer to sleep the MCU for the
 * requested duration. Wakes up via WDT interrupt.
 * This drastically reduces current consumption during idle.
 *
 * The WDT can sleep in 8-second chunks; this function
 * loops until the total duration is achieved.
 */
void enterSleepMode(uint32_t sleep_ms) {
  uint32_t elapsed = 0;

  while (elapsed < sleep_ms) {
    // Select watchdog timeout — use 8s for long sleeps, 1s for short
    uint32_t remaining = sleep_ms - elapsed;
    uint8_t wdtPrescaler;
    uint32_t chunkMs;

    if (remaining >= 8000) {
      wdtPrescaler = (1 << WDP3) | (1 << WDP0);  // 8 seconds
      chunkMs = 8000;
    } else if (remaining >= 4000) {
      wdtPrescaler = (1 << WDP3);  // 4 seconds
      chunkMs = 4000;
    } else if (remaining >= 2000) {
      wdtPrescaler = (1 << WDP2) | (1 << WDP1) | (1 << WDP0);  // 2 seconds
      chunkMs = 2000;
    } else if (remaining >= 1000) {
      wdtPrescaler = (1 << WDP2) | (1 << WDP1);  // 1 second
      chunkMs = 1000;
    } else {
```

```
      // Remainder too small for WDT — use delay
      delay(remaining);
      break;
    }

    // Configure WDT for interrupt (not reset)
    noInterrupts();
    MCUSR &= ~(1 << WDRF);
    WDTCSR |= (1 << WDCE) | (1 << WDE);
    WDTCSR = (1 << WDIE) | wdtPrescaler;
    interrupts();

    // Enter power-down sleep mode
    set_sleep_mode(SLEEP_MODE_PWR_DOWN);
    sleep_enable();
    sleep_cpu();  // CPU halts here until WDT fires
    sleep_disable();

    elapsed += chunkMs;
  }

  // Disable WDT after sleeping
  wdt_disable();
}


/**
 * modeToString()
 * Returns a human-readable string for the current energy mode.
 */
const char *modeToString(EnergyMode mode) {
  switch (mode) {
    case MODE_DEEPSLEEP: return "DEEP_SLEEP";
    case MODE_LOW: return "LOW";
    case MODE_HIGH: return "HIGH";
    case MODE_NORMAL:
    default: return "NORMAL";
  }
}
```

## Laptop Py. Logger_File:

```
import serial
import serial.tools.list_ports
import csv
import argparse
import datetime
import os
import sys
import time


LOG_FILENAME = "sensor_log.csv"
CSV_FIELDS   = ["pc_timestamp", "device_id", "sequence",
                "temperature_C", "humidity_pct", "Vcap_mV",
                "AC_Voltage_V", "energy_mode"]


def list_ports():
    """Print all available serial ports."""
    ports = serial.tools.list_ports.comports()
    if not ports:
        print("No serial ports found.")
    else:
        print("Available serial ports:")
        for p in ports:
            print(f"  {p.device}  —  {p.description}")


def run_logger(port: str, baud: int, log_file: str):
    """Main logging loop."""
    print(f"Connecting to {port} at {baud} baud...")

    try:
        ser = serial.Serial(port, baud, timeout=2)
    except serial.SerialException as e:
        print(f"ERROR: Could not open port {port}: {e}")
        sys.exit(1)

    time.sleep(2)  # Allow Arduino to reset after connection

    # Open CSV file (append mode so data persists across runs)
    file_exists = os.path.exists(log_file)
    csvfile = open(log_file, "a", newline="")
    writer  = csv.DictWriter(csvfile, fieldnames=CSV_FIELDS)

    if not file_exists:
        writer.writeheader()

    print(f"Logging to: {log_file}")
    print("Press Ctrl+C to stop.\n")
    print(f"{'Timestamp':<26} | {'ID':<8} | {'Seq':>6} | {'Temp':>6} |
```

```
{'Hum':>6} | {'Vcap':>6} | {'ACrms':>7} | Mode")
    print("-" * 92)

    try:
        while True:
            raw = ser.readline()
            if not raw:
                continue

            line = raw.decode("utf-8", errors="replace").strip()

            # Skip header / comment lines
            if not line or line.startswith("=") or line.startswith("pc_") or
line.startswith("SYSTEM"):
                if line.startswith("SYSTEM"):
                    print(f"  [NODE STATUS] {line}")
                continue

            # Skip sensor error records
            if line.startswith("SENSOR_ERR"):
                ts = datetime.datetime.now().isoformat(timespec="seconds")
                print(f"  [SENSOR ERROR @ {ts}]  {line}")
                continue

            parts = line.split(",")
            if len(parts) != 7:
                continue  # Malformed line

            ts = datetime.datetime.now().isoformat(timespec="milliseconds")

            row = {
                "pc_timestamp": ts,
                "device_id":    parts[0].strip(),
                "sequence":     parts[1].strip(),
                "temperature_C":parts[2].strip(),
                "humidity_pct": parts[3].strip(),
                "Vcap_mV":      parts[4].strip(),
                "AC_Voltage_V": parts[5].strip(),
                "energy_mode":  parts[6].strip(),
            }
            writer.writerow(row)
            csvfile.flush()

            # Console output
            mode_color = {
                "LOW":       "\033[93m",    # Yellow
                "NORMAL":    "\033[92m",    # Green
                "HIGH":      "\033[96m",    # Cyan
                "DEEP_SLEEP":"\033[91m",    # Red
            }.get(row["energy_mode"], "\033[0m")
            reset = "\033[0m"

            print(
                f"  {ts:<26} | {row['device_id']:<8} | {row['sequence']:>6} |
"
                f"{row['temperature_C']:>5}°C | {row['humidity_pct']:>5}% | "
                f"{row['Vcap_mV']:>5}mV | {row['AC_Voltage_V']:>5}V | "
```

```
                f"{mode_color}{row['energy_mode']}{reset}"
            )

    except KeyboardInterrupt:
        print("\n\nLogging stopped by user.")
    finally:
        ser.close()
        csvfile.close()
        print(f"Data saved to: {log_file}")


def main():
    parser = argparse.ArgumentParser(
        description="Laptop logger for Multi-Source Energy-Harvesting Sensor
Node"
    )
    parser.add_argument("--port",  type=str, default=None,
                        help="Serial port (e.g. COM3 or /dev/ttyUSB0)")
    parser.add_argument("--baud",  type=int, default=9600,
                        help="Baud rate (default: 9600)")
    parser.add_argument("--log",   type=str, default=LOG_FILENAME,
                        help="Output CSV file path")
    parser.add_argument("--list",  action="store_true",
                        help="List available serial ports and exit")
    args = parser.parse_args()

    if args.list:
        list_ports()
        return

    if args.port is None:
        print("ERROR: Please specify --port  (use --list to see available
ports)")
        sys.exit(1)

    run_logger(args.port, args.baud, args.log)


if __name__ == "__main__":
    main()
```